

BUG-001 · Tool-Call Overflow Silently Drops Requests

Field	Detail
Issue	Tool-call overflow — LLM responses exceeding <code>MAX_TOOL_CALLS_PER_ITERATION</code> are silently truncated
Found Date	2026-05 (Batch 9 dry run)
Status	Open
Severity	HIGH
File	<code>agent_query.py</code>
Description	When the LLM generates more tool calls than <code>MAX_TOOL_CALLS_PER_ITERATION</code> (default 5) in one response, the extra calls are silently dropped. The assistant message still contains all tool calls, but only the processed calls receive tool responses. This creates an invalid tool-call history (unmatched <code>tool_call_ids</code>) and can silently discard retrieval calls or the <code>compress_context</code> call. Observed in Q57 where 7 tool calls were generated (6 retrievals + compress), but only 5 were processed.
Root Cause	<code>resp_tool_calls[:MAX_TOOL_CALLS_PER_ITERATION]</code> slice without trimming the assistant message or providing stub responses for skipped call IDs.
Solution	Either (a) trim <code>assistant.tool_calls</code> to the same slice before appending to messages, or (b) append "skipped due to cap" tool responses for every unprocessed <code>tool_call_id</code>.
Date Resolved	—

BUG-002 · Retrieval Judge Over-Lenient on Bibliography Chunks

Field	Detail
Issue	<code>validate_retrieval</code> passes bibliography/citation-list chunks as relevant
Found Date	2026-05 (Batch 9 dry run)
Status	Open
Severity	MEDIUM
File	<code>validators.py</code>
Description	Chunks that are pure reference lists (e.g., "Vaccine 2014;32:3623-9") are being judged as PASS by the retrieval validator. This pollutes the compression pipeline with citation metadata instead of substantive facts, degrading answer quality. Confirmed in Q57 (prevalence query) and Q4 (vaccines).
Root Cause	The retrieval judge prompt has no rule to identify bibliography-only chunks as irrelevant.
Solution	Add a rule to <code>validate_retrieval</code> : if a chunk consists primarily of citations, author names, and publication references with no substantive factual sentences, return IRRELEVANT unless the user query is explicitly about references or bibliography.
Date Resolved	—

BUG-003 · DC Redundancy Judge Creates False Positives (Deletes Non-Duplicate Content)

Field	Detail
Issue	Deduplication Compression (DC) marks semantically distinct sentences as redundant and removes them
Found Date	2026-05 (Batch 9 dry run)
Status	Open
Severity	MEDIUM

File	context_compression.py, validators.py
Description	In Q57, the DC stage treated "gender and racial factors shape who is recognized..." as redundant with "providers should be trained to recognize masking..." and removed content. These sentences carry different factual claims. The validate_redundancy judge is confirming groups where members are topically adjacent but not semantically entailing each other.
Root Cause	The DC validator uses topic similarity rather than semantic entailment to confirm redundancy. A "confirmed" verdict should require both sentences to convey identical information in both directions.
Solution	Make DC deletion require exact semantic entailment both ways. Alternatively, disable LLM sentence deletion unless the sentence match is high-confidence by embedding similarity + NLI/LLM judge. Raise the confirmation threshold significantly.
Date Resolved	—

BUG-004 · LBC Can Expand Chunks Instead of Compressing

Field	Detail
Issue	LLM-Based Compression (LBC) produces output longer than its input, causing negative reduction
Found Date	2026-05 (Batch 9 dry run)
Status	Open
Severity	MEDIUM
File	context_compression.py
Description	In Q57, LBC produced compressed chunks that expanded badly: 367 → 1134 chars and 170 → 2130 chars. Total context went from 5,488 → 7,471 chars, reported as negative reduction. The existing over-compression guard (blocks output < 35% of input) does not catch over-expansion.
Root Cause	No over-expansion guard. The LBC stage only checks that compression isn't too aggressive; it doesn't check that the LLM

	hasn't padded or elaborated the content.
Solution	Add guard: if len(compressed_text) > len(original_content): keep original. This is a one-line fix in <code>llm_based_compression</code> .
Date Resolved	—

BUG-005 · check_answer_quality Judge Blind to Semantic Extension / Hallucination

Field	Detail
Issue	<code>check_answer_quality</code> passes answers that hallucinate facts not in retrieved chunks
Found Date	2026-04 (early dry runs), recurring across Batches 5–9
Status	Open
Severity	CRITICAL
File	<code>tools.py</code>
Description	The CAQ judge verifies grounding (does the answer follow from retrieved text?) but not scope (did the retrieved text actually cover the question domain?). Multiple confirmed false-positive verdicts: Q72 — LLM substituted "medical equipment" for "sensitive equipment" (source never mentioned medical); Q73 — fabricated longitudinal outcome studies; Q70 — fabricated "mathematics or music" exceptional abilities; Q5 — invented therapy names (ABA, OT, Speech Therapy) not in any chunk; Q22 — hallucinated 2023 CDC ADDM report details using 2016 data.
Root Cause	Quality check uses word-overlap heuristic or a loose LLM judge, neither of which detects semantic extension — where the LLM adds plausible-sounding facts adjacent to the topic that no chunk states.
Solution	(1) Add a named-entity grounding check: every named entity in the answer must appear in a retrieved chunk. (2) Add a temporal grounding check: if the query specifies a year/report, retrieved chunks must match that time.

	(3) Detect context-length mismatch: if <code>len(answer) > len(context) * 3</code> and <code>len(context) < 200</code>, flag as suspicious. (4) Strengthen minimum context-length threshold from 50 to 100 chars.
Date Resolved	—

BUG-006 · No Dataset/Schema Awareness — Data Interpretation Questions Produce Clinical Hallucinations

Field	Detail
Issue	System answers CSV/dataset-schema questions using clinical knowledge, producing wrong answers
Found Date	2026-05 (Batch 9 dry run, Q71)
Status	Open
Severity	HIGH
File	<code>agent_query.py</code> , <code>retriever.py</code>
Description	Q71 asked why <code>ASD_Diagnosis=No</code> patients have <code>ASD_Severity=NaN</code>. The correct answer is "no diagnosis means no severity is applicable — this is expected dataset behavior." Instead, the system retrieved DSM-5 clinical criteria chunks and fabricated a clinical explanation about diagnostic specificity. The <code>check_answer_quality</code> judge passed this as OK.
Root Cause	The knowledge base contains no document describing the CSV schema or the logical relationship between dataset columns. When no relevant schema chunks are retrieved, the LLM bridges the gap with plausible clinical content instead of triggering <code>NO_CONTEXT_ANSWER</code>.
Solution	(1) Add a schema/data-dictionary document to the knowledge base. (2) Detect data-schema questions by pattern (column name references, <code>NaN</code>, data type questions) and route them to a dedicated handler that checks the dataset metadata first. (3) Lower the CAQ threshold for this question category.

Date Resolved	—
----------------------	---

BUG-007 · Retrieval Near-Synonym Loop Not Programmatically Enforced

Field	Detail
Issue	LLM generates semantically identical queries across iterations, burning the retrieval budget
Found Date	2026-05 (Batches 3, 9; confirmed in Q73)
Status	Open
Severity	MEDIUM
File	agent_query.py
Description	In Q73, the agent used all 6 iterations and all 5 retrieval slots, with 4 out of 5 queries being near-synonyms: "early diagnosis ASD outcomes", "ASD diagnosis timing impact", "impact of delayed ASD diagnosis on long-term outcomes", "consequences of delayed ASD diagnosis on quality of life." The system prompt says "Never generate two queries with identical words" but this only catches lexical overlap; semantic duplicates slip through.
Root Cause	No programmatic deduplication of queries by semantic similarity. The constraint is self-regulated by the LLM, which ignores it under retrieval pressure.
Solution	Add an embedding-based pre-check before dispatching each queued query: if cosine similarity against any prior query in this turn is above ~0.85, skip it and log [REDUNDANT VARIANT] skipped. Infrastructure for [FAILED VARIANT] recorded already exists and can be extended.
Date Resolved	—

BUG-008 · Confidence Field Self-Reported by LLM, Not Grounded in Metrics

Field	Detail
Issue	Answer confidence is self-assessed by the LLM rather than computed from retrieval quality
Found Date	2026-04 (early dry runs, Q3 Group 2)
Status	Open
Severity	MEDIUM
File	agent_query.py
Description	The LLM reports HIGH confidence even when it has essentially no real retrieved data. In one case it reported "HIGH — The knowledge base contains relevant documents" while producing a near-complete hallucination. The confidence field backfired: the model fills it based on its own parametric certainty, not actual retrieval quality.
Root Cause	Confidence is prompted as a free-text field in the output format, allowing the LLM to self-assess.
Solution	Compute confidence programmatically before returning the answer: avg_score and total_chars of retrieved chunks → HIGH/MEDIUM/LOW. Never let the LLM self-report it.
Date Resolved	—

BUG-009 · Thumbdown Query Matching Requires Exact String Equality

Field	Detail
Issue	Thumbdown feedback does not carry over to semantically equivalent follow-up queries
Found Date	2026-05 (Batch 1, Q7/Q8)
Status	Open

Severity	LOW
File	<code>feedback_store.py</code>
Description	If a user thumbs-down "What is ASD?" and later asks "What is ASD and what are its main characteristics?", these are different normalized strings and the thumbdown does not transfer. The pivot/retry mechanism is never triggered for semantically equivalent but lexically different queries.
Root Cause	Feedback lookup uses exact string normalization, not semantic similarity.
Solution	Implement fuzzy or embedding-based thumbdown matching. Compare new queries against the stored thumbdown set using cosine similarity above a threshold (e.g. 0.85).
Date Resolved	—

BUG-010 · Coverage Checker Too Weak for Multi-Part Queries

Field	Detail
Issue	<code>check_answer_quality</code> returns OK even when only some sub-questions of a multi-part query are answered
Found Date	2026-05 (Batch 9, Q58)
Status	Open
Severity	MEDIUM
File	<code>tools.py</code>
Description	Q58 asked for clinical criteria, patient characteristics, management strategies, and diagnostic impact. The final answer mostly covered patient traits and management but barely addressed clinical criteria or diagnostic impact. CAQ still returned OK.
Root Cause	The quality judge does not decompose the user-query into sub-questions and verify each one is answered.

Solution	Add a coverage checklist: extract explicit sub-questions or facets from the user query (via LLM or heuristic), require each to be addressed before returning OK.
Date Resolved	—

BUG-F001 · Merge LLM JSON Escaping Bug (Windows Backslash Paths)

Field	Detail
Issue	<code>merge_similar_chunks</code> fails on Windows file paths with backslashes in JSON strings
Found Date	2026-04 (Batch 1, Q1)
Status	Closed
Severity	HIGH
File	<code>context_compression.py</code> (formerly in <code>agent_query.py</code>)
Description	The chunk-merge LLM returned JSON where source paths used unescaped Windows backslashes inside double-quoted strings (e.g., "C:\Users\...") causing <code>json.loads()</code> to throw <code>JSONDecodeError</code> . Fallback returned only <code>similar_chunks[0]</code> , silently discarding other chunks.
Root Cause	LLM inconsistently quoted Windows paths inside JSON strings; JSON requires inner double quotes to be escaped as <code>"\"</code> .
Solution	Normalize source paths to forward slashes before passing to the merge LLM. Apply path normalization at ingest time and during chunk-merge JSON serialization.
Date Resolved	2026-05 (confirmed fixed in Batch 2 log)

BUG-F002 · OPENAI_BASE_URL Resolved at Module Import Time Before load_dotenv()

Field	Detail
Issue	<code>OPENAI_BASE_URL</code> evaluated at module scope before <code>.env</code> was loaded, causing silent fallback to OpenAI's real endpoint and a 401 error
Found Date	2026-05
Status	Closed
Severity	HIGH
File	<code>agent_query.py</code>
Description	A patch added <code>OPENAI_BASE_URL = normalize_openai_base_url(os.getenv("CUSTOM_API_BASE"))</code> at module top-level. But <code>load_dotenv()</code> only runs inside <code>main()</code> . At the time the module-level line executed, <code>CUSTOM_API_BASE</code> was <code>None</code> . <code>ChatOpenAI(base_url=None)</code> fell back to <code>api.openai.com</code> , causing 401 rejections with the custom API key.
Root Cause	Module-level constant resolved before environment was populated.
Solution	Move <code>api_key</code> , <code>api_base</code> , and <code>model</code> resolution inside <code>main()</code> , immediately after <code>load_dotenv()</code> . Added explicit <code>RuntimeError</code> if either env var is missing. Added <code>[LLM] endpoint=...</code> startup log line as a sanity check.
Date Resolved	2026-05-14

**BUG-F003 · CUSTOM_API_BASE Double-Appends /chat/completions
→ 404**

Field	Detail
Issue	API calls returned 404 because LangChain appended /chat/completions to a base URL that already ended with /chat/completions
Found Date	2026-05-14
Status	Closed
Severity	HIGH
File	agent_query.py
Description	The .env value for CUSTOM_API_BASE was the full endpoint (http://host:port/v1/chat/completions), which test_run.py POSTs to directly. ChatOpenAI treats base_url as a root and appends /chat/completions itself, resulting in .../chat/completions/chat/completions → 404.
Root Cause	Two consumers (requests.post in test_run.py and LangChain's ChatOpenAI) had incompatible expectations of the base_url format.
Solution	Added normalize_openai_base_url() helper that strips trailing /chat/completions, /completions, and /responses suffixes before passing to ChatOpenAI. Function is idempotent — already-correct root URLs are unchanged.
Date Resolved	2026-05-14

BUG-F004 · Local Server Returns `function.arguments` as Dict Instead of JSON String

Field	Detail
Issue	HuggingFace TGI server returned tool call <code>arguments</code> as a JSON object instead of a JSON-encoded string, causing Pydantic validation failure in LangChain
Found Date	2026-05-14
Status	Closed
Severity	HIGH
File	<code>agent_query.py</code> (HTTP client layer)
Description	OpenAI spec requires <code>function.arguments</code> to be a JSON-encoded string (e.g., <code>"{"location": "Lahore"}"</code>). The local TGI server returned it as a raw dict object. LangChain's response-parsing layer raised a Pydantic validation error before the application code could see the message.
Root Cause	TGI server non-compliance with OpenAI tool-call response format.
Solution	Intercepted the HTTP response via a custom <code>httpx.Client</code> hook before LangChain's Pydantic parsing. Coerced <code>arguments</code> to <code>json.dumps(arguments)</code> if it arrived as a dict. Wired via <code>ChatOpenAI(http_client=build_tolerant_http_client())</code> .
Date Resolved	2026-05-14

BUG-F005 · DC Stage Silently Fails JSON Parse and Treats Truncation as "No Duplicates"

Field	Detail
Issue	Truncated DC scan JSON is silently treated as zero redundancies found
Found Date	2026-05 (Batch 1, Q5)

Status	Closed
Severity	MEDIUM
File	context_compression.py
Description	The LLM-generated JSON for the dedup scan was truncated mid-output. The pipeline caught the <code>JSONDecodeError</code> and silently coerced to "0 redundant passages found," leaving duplicate content in context. This happened twice in the same compression call (windows <code>[0:3]</code> and <code>[3:5]</code>).
Root Cause	<code>max_tokens</code> on the DC scan call was too low for larger windows; truncation was not distinguished from a legitimate empty-list response.
Solution	Increased <code>max_tokens</code> on the DC scan call. Added greedy regex fallback to recover partial JSON arrays. Added explicit log warning on parse failure rather than silent coercion. The <code>fix_llm_output.py</code> module now handles multiple JSON repair tiers.
Date Resolved	2026-05

BUG-F006 · DC Judge Outputs Python Code Instead of JSON

Field	Detail
Issue	<code>llama-3.1-8b-instant</code> responded to the redundancy-scan prompt with Python code instead of a JSON array
Found Date	2026-05 (Batch 1, Q5; recurring)
Status	Closed
Severity	MEDIUM
File	context_compression.py
Description	The DC scan prompt contained words like "detect," "filter," "redundant" which the small model pattern-matched as a coding task, generating <code>def detect_redundancy(chunks):</code> instead of the required JSON output. The safe fallback (treat all groups as

	REJECTED) triggered, meaning no deduplication ran.
Root Cause	Small instruction-tuned models (8B class) pattern-match on programming-adjacent verbs in prompts and respond with code.
Solution	Rewrote the DC scan prompt to remove all programming vocabulary. Added explicit instruction: "Your response must be a JSON array only. Do not write code." Added <code>fix_llm_output.py</code> with a balanced-bracket JSON extraction tier to recover JSON literals embedded in Python code as a fallback.
Date Resolved	2026-05

BUG-F007 · `compression.py` Shadows Python 3.14 Standard Library Module

Field	Detail
Issue	Module named <code>compression.py</code> in <code>app/</code> shadowed Python 3.14's new <code>compression</code> stdlib package, causing circular import crash on startup
Found Date	2026-05-13
Status	Closed
Severity	CRITICAL
File	<code>compression.py</code> → renamed to <code>context_compression.py</code>
Description	Python 3.14 introduced a stdlib package named <code>compression</code> (used by <code>bz2</code> , <code>gzip</code>). Because <code>app/</code> was on <code>sys.path</code> ahead of the stdlib, <code>bz2</code> did <code>from compression import streams</code> and found the project file instead. This triggered a circular import chain through <code>tempfile</code> → <code>shutil</code> → <code>bz2</code> and caused <code>AttributeError: partially initialized module 'tempfile' on every startup</code> .
Root Cause	File naming conflict with a new Python 3.14 stdlib module.
Solution	Renamed <code>compression.py</code> → <code>context_compression.py</code> . Updated all import statements in <code>tools.py</code> and <code>agent_query.py</code> .

Date Resolved	2026-05-13
----------------------	------------

BUG-F008 · Early COMPRESS Trigger Fires After First Retrieval, Preventing Multi-Query Retrieval

Field	Detail
Issue	Orchestrator forced COMPRESS phase after just 1 retrieval, preventing the LLM from issuing its 2nd and 3rd queries
Found Date	2026-05-18
Status	Closed
Severity	HIGH
File	<code>agent_query.py</code>
Description	Guard <pre>if total_retrievals >= 1 and agent_state.get("accumulated_chunks"): phase = "COMPRESS"</pre> fired immediately after the first successful retrieval. This meant only 1 chunk from 1 source was ever retrieved, resulting in hallucinated answers drawn from the LLM's parametric memory. Old runs used 3 retrievals → 3 sources → full NAC/DC/LBC pipeline. New runs used 1 retrieval → 1 source → skipped compression (below 500-token threshold).
Root Cause	Guard was added to reduce unnecessary LLM round-trips but was too aggressive — fired unconditionally after any single retrieval.
Solution	Added <code>MIN_RETRIEVALS_BEFORE_COMPRESS = 2</code> constant. Changed the early-COMPRESS guard to only trigger when <code>total_retrievals >= MIN_RETRIEVALS_BEFORE_COMPRESS</code> . Below that, the loop stays in RETRIEVE phase and allows sequential tool calls from smaller models.
Date Resolved	2026-05-18

BUG-F009 · json-repair Not in requirements.txt — Repair Tier Silently Offline

Field	Detail
Issue	<code>fix_llm_output.py</code> repair tier silently disabled when <code>json-repair</code> package was not installed
Found Date	2026-05-21
Status	Closed
Severity	MEDIUM
File	<code>fix_llm_output.py</code> , <code>requirements.txt</code>
Description	<code>fix_llm_output.py</code> has a <code>try/except ImportError</code> that sets <code>HAS_JSON_REPAIR = False</code> and degrades gracefully — but silently. 7/99 test cases failed with no obvious error, all on cases requiring the repair tier (trailing commas, single quotes, missing commas, truncated JSON). Root cause was only identified after investigation: <code>json-repair</code> was not in <code>requirements.txt</code> and not installed.
Root Cause	Missing dependency declaration.
Solution	Added <code>json-repair >= 0.30</code> to <code>requirements.txt</code> / <code>pyproject.toml</code> via <code>uv add json-repair</code> . Patched <code>test_output_fixes.py</code> to emit a visible warning when <code>HAS_JSON_REPAIR = False</code> . Added import-time warning to <code>fix_llm_output.py</code> itself.
Date Resolved	2026-05-21

BUG-F010 · Inconsistent Source Path Formatting Pollutes `learned_qa` Store

Field	Detail
Issue	Mixed forward/back slashes in source citation paths cause false negatives when deduplicating the <code>learned_qa</code> store
Found Date	2026-05 (Batch 1, Q1)

Status	Closed
Severity	LOW
File	context_compression.py, self_learner.py
Description	Retriever stored Windows-style paths (\\data\\pdfs\\...). The merge LLM sometimes rewrote them as forward-slash paths (./data/pdfs/...) and sometimes left them mixed. Result: learned_qa stored non-canonical source strings; downstream code deduplicating by source path would see false negatives.
Root Cause	No single normalization point — paths were written at ingest with OS separators, then re-processed inconsistently at merge time.
Solution	Applied str.replace("\\", "/") normalization at ingest time and again in the chunk-merge serialization step. All source paths now stored with forward slashes.
Date Resolved	2026-05

BUG-F011 · DRAFT Phase Sent Full Tool-Call History to Local Server → HTTP 500

Field	Detail
Issue	DRAFT phase passed the full messages history (containing prior tool calls) to the local inference server, causing HTTP 500
Found Date	2026-05 (local server testing)
Status	Closed
Severity	HIGH
File	agent_query.py
Description	The DRAFT LLM call used llm.invoke(llm, messages, ...) where messages contained the full conversation history including all retrieval tool calls and results. The local TGI server crashed (HTTP 500) when receiving this history. Retrieval and compression phases succeeded; only DRAFT

	failed.
Root Cause	Local server does not handle tool-call history in a stateless completion request.
Solution	Made DRAFT call stateless: replaced <code>messages</code> with a fresh <code>draft_messages</code> list containing only a system prompt and a single user message with <code>USER QUERY + RETRIEVED CONTEXT</code> . Removed the <code>messages.pop()</code> cleanup that was no longer needed.
Date Resolved	2026-05

BUG-F012 · `BadRequestError` / `tool_use_failed` from Groq API on Critical Iterations

Field	Detail
Issue	Groq API rejected tool-call payload mid-session with <code>BadRequestError</code> , causing agent to exit with "Unable to generate a clean answer"
Found Date	2026-05 (Batch 1, Q2)
Status	Closed
Severity	HIGH
File	<code>llm_caller.py</code> , <code>agent_query.py</code>
Description	The Groq API returned <code>tool_use_failed</code> at a critical moment, and the fallback stripped non-tool content and exited immediately. No retry, no graceful fallback answer, just a hard abort.
Root Cause	No retry logic for transient <code>BadRequestError</code> / <code>tool_use_failed</code> ; fallback was too aggressive (hard exit).
Solution	Implemented <code>LLMErrorKind</code> taxonomy in <code>llm_caller.py</code> (<code>RATE_LIMIT</code> , <code>SERVER_ERROR</code> , <code>CONNECTION</code> , <code>TIMEOUT</code> , <code>BAD_REQUEST</code> , etc.). Added transient-error retry loop with exponential backoff (1s, 2s, 4s, 8s, max 4 retries) for recoverable error kinds. Non-retryable errors fail

	fast with a structured log.
Date Resolved	2026-05-11

BUG-F013 · Runaway Agent Loop (40+ Iterations on Single Query)

Field	Detail
Issue	Agent looped 40+ times on a single query, burning through Groq's 6,000 TPM rate limit
Found Date	2026-04 (early dry runs)
Status	Closed
Severity	HIGH
File	agent_query.py
Description	With no guardrails on per-iteration tool calls, the agent generated increasingly redundant queries across 40+ iterations. The accumulated chunks in <code>messages</code> triggered Groq's TPM limit (413: Requested 6785, Limit 6000).
Root Cause	No iteration cap, no per-turn query deduplication, no context-size pruning.
Solution	Added <code>MAX_ITERATIONS = 6</code> , <code>MAX_TOOL_CALLS_PER_ITERATION = 5</code> , <code>MAX_TOTAL_RETRIEVALS = 5</code> . Added in-batch dedup check (<code>seen_queries</code> set). Refactored compression pipeline to replace raw chunk messages with a single formatted context, preventing context window explosion.
Date Resolved	2026-04

BUG-F014 · Distillation Overgeneralizes Mouse-Model Study as Human Causal Evidence

Field	Detail
Issue	Self-learning distillation stored a QA pair that overstates a mouse-model study as evidence of a causal vaccine–autism link

Found Date	2026-05 (Batch 1, post-Q6 distillation run)
Status	Closed
Severity	LOW
File	self_learner.py
Description	After Q6, the distillation run processed a citation-fragment chunk about maternal immune activation (mouse model) and produced a learned QA pair that overgeneralized it as evidence of a causal link to autism in humans. The pair was stored in learned_qa.
Root Cause	Distillation LLM prompt did not require the model to stay within the evidential scope of the source chunk (animal vs. human study distinction).
Solution	Added evidential-scope constraint to the distillation prompt: require the LLM to faithfully represent the evidence type (animal model, observational, RCT, meta-analysis) and prohibit extrapolation across study types.
Date Resolved	2026-05

BUG-015 · GROQ_API_BASE Env Var Causes Double-Path URL in Auto-Instantiated ChatGroq

Field	Detail
Issue	Auto-instantiated ChatGroq objects in LLM_Json_Repair and Verify_And_Correct send requests to /openai/v1/openai/v1/chat/completions, resulting in HTTP 404 on every LLM repair call
Found Date	2026-06-08
Status	Closed
Severity	HIGH
File	fix_llm_output.py — LLM_Json_Repair(), _Verify_And_Correct()

Description	When <code>.env</code> contains <code>GROQ_API_BASE=https://api.groq.com/openai/v1</code> , <code>langchain_groq</code> passes this directly to the underlying <code>groq</code> SDK as <code>base_url</code> . The SDK then appends its standard path suffix to that value, doubling the <code>/openai/v1</code> segment. Observed in <code>run_logs/llm_data_check.txt</code> : Error code: 404 — Unknown request URL: POST /openai/v1/openai/v1/chat/completions.
Root Cause	<code>.env</code> sets <code>GROQ_API_BASE</code> for a different integration context. <code>langchain_groq</code> passes the full value verbatim as <code>base_url</code> ; the <code>groq</code> SDK then appends its standard endpoint path suffix, producing a doubled path.
Solution	<code>os.environ.pop("GROQ_API_BASE", None)</code> immediately before <code>ChatGroq(...)</code> in both auto-instantiation blocks. The <code>groq</code> SDK defaults to <code>https://api.groq.com</code> when <code>base_url</code> is unset, which is the correct base.
Date Resolved	2026-06-08

BUG-016 · `_LLM_Json_Repair` Log File Never Written — Write Block Was Unreachable

Field	Detail
Issue	<code>run_logs/llm_json_tries.txt</code> was never created even when LLM repair calls were being made
Found Date	2026-06-08
Status	Closed
Severity	HIGH
File	<code>fix_llm_output.py</code> — <code>_LLM_Json_Repair()</code>
Description	The <code>os.makedirs / open</code> log-write block was placed after early-return guards (<code>if llm is None: return None</code> and <code>if not result.ok</code>), making it unreachable in all normal execution paths. The function returned before the log block could execute regardless of whether the LLM call succeeded or failed.
Root Cause	Log write was placed inside a conditional branch rather than at

	the function's single exit point.
Solution	Restructured <code>LLM_Json_Repair</code> so, the log write executes unconditionally at the function's end. Runs that skip the LLM call write a <code>skipped: no LLM provided</code> entry, so the file is created on the very first call, confirming the logging path is live.
Date Resolved	2026-06-08

BUG-017 · `fix_llm_output` Returned Failure for Every Successful Parse — Inverted Guard Condition

Field	Detail
Issue	Majority of <code>test_output_fixes.py</code> cases were failing immediately — <code>fix_llm_output</code> returned <code>(_empty, False)</code> even when <code>_parse_to_python</code> returned a valid object
Found Date	2026-06-08
Status	Closed
Severity	HIGH
File	<code>fix_llm_output.py</code> — <code>fix_llm_output()</code>
Description	The early-exit guard after <code>_parse_to_python</code> was written as <code>if obj is not None: return _empty, False</code> — exactly inverted. This caused the function to return the empty fallback precisely when parsing succeeded, and to continue processing only when parsing failed. Test pass rate was near-zero immediately after the first build.
Root Cause	Logic inversion in the original stub — <code>is not None</code> instead of <code>is None</code> .
Solution	Corrected to <code>if obj is None: return _empty(top_level), False</code> .
Date Resolved	2026-06-08

BUG-018 · 12 Test Case Expected-Outcome Flags Not Updated After LLM Repair Was Added

Field	Detail
Issue	12 test cases in <code>test_output_fixes.py</code> had <code>expect_correct=False</code> but were now passing after LLM repair was added — causing false test failures
Found Date	2026-06-08
Status	Closed
Severity	MEDIUM
File	<code>test_output_fixes.py</code>
Description	Test expectations were written for the pre-LLM-repair pipeline. After <code>LLM_Json_Repair</code> was added, cases involving XML output, YAML output, PascalCase key remapping, dataclass syntax, and nested object unwrapping were correctly reconstructed by the LLM — but still marked <code>expect_correct=False</code> , so the test harness reported them as failures even though the code was behaving correctly.
Root Cause	Test expectations not updated when the pipeline's capability set changed.
Solution	Updated 12 cases to <code>expect_correct=True</code> : C14-merge (nested unwrap), C23-merge (XML), C24-merge (YAML), C32-merge (PascalCase keys), PC02-merge (dataclass syntax), C36-rj (partial hallucination unwrap), C32-rj (PascalCase keys), PC04-rj/mj/lc/lj/dc (class attribute syntax). Cases kept <code>False</code> : refusals, pure functions with no data, missing required fields with no source data in the raw text, null required fields, set/comprehension/f-string/lambda syntax — all cases where the LLM would have to fabricate rather than reconstruct.
Date Resolved	2026-06-08

BUG-019 · _Verify_And_Correct Hallucinates Source Filenames for Empty sources Fields

Field	Detail
Issue	The value-verification LLM occasionally populates <code>sources: []</code> with invented noun phrases when no filename appears in the raw response
Found Date	2026-06-08
Status	Open
Severity	LOW
File	<code>fix_llm_output.py</code> — <code>_VALUE_VERIFY_PROMPT</code>
Description	Observed in <code>run_logs/llm_data_check.txt</code> : for inputs where <code>sources</code> is empty and no source filenames (<code>*.pdf</code> , <code>*.txt</code>) appear in the raw response, <code>llama-3.1-8b-instant</code> at <code>temperature=0.0</code> sometimes infers a source from noun phrases in the content (e.g. <code>sources: ["the retrieved chunks"]</code>). The output is structurally valid but fabricated — no actual filename was present in the raw text.
Root Cause	<code>_VALUE_VERIFY_PROMPT</code> instructs the LLM to use existing defaults for genuinely missing fields, but does not give explicit guidance on how to identify a "source filename" field vs a general string field. The model treats it as a string it can populate from context.
Solution	Add an explicit rule to <code>_VALUE_VERIFY_PROMPT</code> : "If a field contains source filenames (e.g. <code>sources</code>) and no string matching a filename pattern (ending in <code>.pdf</code> , <code>.txt</code> , <code>.docx</code> , etc.) appears in the RAW RESPONSE, set it to <code>[]</code> ." Monitor <code>correction_applied</code> rate in <code>llm_data_check.txt</code> after the change.
Date Resolved	—

BUG-020 · Validators Crash on 'str' object has no attribute 'get' When *_OUTPUT_FIX Flag Is Off

Field	Detail
Issue	Every LLM-as-judge validator (<code>validate_retrieval</code> , <code>validate_merge</code> , <code>validate_redundancy</code> , <code>validate_lbc</code>) crashes with <code>AttributeError: 'str' object has no attribute 'get'</code> when its per-stage *_OUTPUT_FIX flag or the global <code>ENABLE_GLOBAL_LLM_OUTPUT_FIX</code> flag is off
Found Date	2026-06-09 (<code>run_combinations.py</code> ladder runs 3–9)
Status	Open
Severity	HIGH
File	<code>validators.py</code> — affects <code>validate_retrieval</code> (line 134), <code>validate_merge</code> (line 252), and the same fall-through pattern in <code>validate_redundancy</code> and <code>validate_lbc</code>
Description	Each validator has a two-branch structure: if both the per-validator <code>OUTPUT_FIX</code> flag and the global flag are on, it calls <code>fix_llm_output(...)</code> and the result is a dict; otherwise the else branch executes <code>llm_result = raw</code> — assigning the raw LLM response string directly. The very next line then calls <code>llm_result.get("verdict", ...)</code> (or <code>.get("fabricated_claims", ...)</code> in the merge validator) which crashes because strings have no <code>.get()</code> method. Confirmed in ladder runs 3 (<code>validate_merge</code>), 4–9 (<code>validate_retrieval</code>): each crashed at iteration 1 of the RETRIEVE phase the moment the first validator fired with its output-fix flag off. Runs 1, 2 (no validators enabled), and 10 (everything on) completed. There is also a minor logging bug in the success branch — the print line reads "failed to fix malformed LLM output" when fix succeeded.
Root Cause	The else branch assumes the validator can consume the raw LLM string directly, but the downstream code is dict-only. The branch was a hold-over from before <code>fix_llm_output</code> was introduced and was never updated to call <code>json.loads(raw)</code> (or <code>safe_json_parse</code>) as a minimum-effort fallback.
Solution	In each validator's else branch, replace <code>llm_result = raw</code> with a <code>json.loads(raw)</code> call wrapped in a try/except that returns the <code>verdict="UNKNOWN"</code> empty-result on failure (mirroring the

	same return path used when <code>fix_llm_output</code> fails). Audit all four validators for this pattern. Also fix the inner-else log message to say "successfully parsed LLM output" instead of "failed to fix".
Date Resolved	—

BUG-021 · ENABLE_AUTO_DISTILLATION Flag Defined but Never Read

Field	Detail
Issue	<code>ENABLE_AUTO_DISTILLATION</code> is exported from <code>config.py</code> and toggled by <code>run_combinations.py</code> , but <code>agent_query.py</code> never imports or checks it — distillation fires unconditionally on <code>should_learn()</code> regardless of the flag
Found Date	2026-06-09 (post-ladder code review)
Status	Open
Severity	MEDIUM
File	<code>agent_query.py</code> (callsites at lines 1045 and 1128 around <code>self_learner.run_distillation()</code>)
Description	The ladder harness toggles all 18 flags in <code>BOOL_FLAGS</code>, including <code>ENABLE_AUTO_DISTILLATION</code>. The flag's intended effect is to gate the automatic <code>run_distillation()</code> calls that fire after every <code>LEARN_EVERY_N</code> successful interactions. However, <code>agent_query.py</code> does not import <code>ENABLE_AUTO_DISTILLATION</code> from <code>config</code>, and the two distillation callsites have no <code>if ENABLE_AUTO_DISTILLATION:</code> guard. Result: ladder runs that should be measuring the pipeline without distillation still trigger distillation, polluting the <code>learned_qa</code> collection across runs and making per-step ladder comparisons noisier than intended. The ladder harness toggles all 18 flags in <code>BOOL_FLAGS</code>, including <code>ENABLE_AUTO_DISTILLATION</code>. The flag's intended effect is to gate the automatic <code>run_distillation()</code> calls that fire after every <code>LEARN_EVERY_N</code> successful interactions. However, <code>agent_query.py</code> does not import <code>ENABLE_AUTO_DISTILLATION</code> from <code>config</code>, and the two distillation callsites have no <code>if</code>

	<code>ENABLE_AUTO_DISTILLATION</code> : guard. Result: ladder runs that should be measuring the pipeline without distillation still trigger distillation, polluting the <code>learned_qa</code> collection across runs and making per-step ladder comparisons noisier than intended.
Root Cause	Flag was added to <code>config.py</code> during the centralisation pass but the corresponding <code>if ENABLE_AUTO_DISTILLATION</code> : guards were not added at the two distillation callsites in <code>agent_query.py</code> .
Solution	Import <code>ENABLE_AUTO_DISTILLATION</code> from <code>config</code> in <code>agent_query.py</code> and wrap both <code>self_learner.run_distillation()</code> callsites with <code>if ENABLE_AUTO_DISTILLATION</code> :. Verify by re-running the ladder and confirming the <code>learned_qa</code> count stays constant for steps where the flag is off.
Date Resolved	—

BUG-022 · Raw LLM Responses Used When Output Repair Disabled

Field	Detail
Issue	Multiple validator and compression fallback paths assumed structured Python objects even when <code>ENABLE_GLOBAL_LLM_OUTPUT_FIX</code> (or stage-specific output-fix flags) was disabled. Raw LLM responses could reach downstream code that immediately called <code>.get()</code> , iterated results, or accessed expected JSON fields.
Status	Resolved
Severity	HIGH
File	<code>validators.py</code> , <code>context_compression.py</code> , and other fallback parsing paths that bypass <code>fix_llm_output()</code>
Description	The newly introduced configuration architecture allows the output-repair layer to be disabled independently. Dry runs revealed that several fallback branches still assumed repaired JSON output and attempted to operate directly on raw LLM strings. Depending on the response shape, this produced failures such as <code>AttributeError: 'NoneType' object has no attribute 'get'</code> , <code>AttributeError: 'list' object has no attribute 'get'</code> , and iteration-related crashes. As a result, disabling output repair could break otherwise functional retrieval, validation, merge, and compression flows.

Root Cause	The original implementation was written assuming all structured LLM outputs would pass through <code>fix_llm_output()</code> . When the output-repair flags were later added, fallback branches were introduced but were not updated to perform JSON parsing before downstream consumption.
Solution	Updated fallback paths to route responses through <code>parse_to_python()</code> whenever output repair is disabled. This ensures validators and compression stages still receive parsed Python objects instead of raw LLM text while preserving the ability to disable the repair layer for experimentation and ladder testing.
Date Resolved	2026-06-10

BUG-023 · Shared `_merge_similar_chunks()` Logic Created Cross-Pipeline Coupling

Field	Detail
Issue	<code>_merge_similar_chunks()</code> was being used by both retrieval deduplication/merge and Neighbor-Aware Compression (NAC), but recent modifications assumed a single execution path.
Found Date	2026-06-10 (during configuration-combination dry runs)
Status	Resolved
Severity	MEDIUM
File	<code>context_compression.py</code>
Description	While validating feature combinations, inconsistent behavior appeared depending on whether retrieval deduplication or NAC compression invoked <code>_merge_similar_chunks()</code> . The function had effectively become shared infrastructure, but some logic changes only considered one caller. This created subtle execution differences when specific pipeline stages were enabled or disabled.
Root Cause	Architectural coupling emerged because the same merge implementation serves multiple subsystems. The merge code evolved as if it belonged to a single feature area, despite being

	reused across distinct retrieval and compression flows.
Solution	Updated merge handling to correctly support both retrieval deduplication and NAC execution paths. Future modifications should treat <code>_merge_similar_chunks()</code> as shared infrastructure and validate both callers during testing.
Date Resolved	2026-06-10

BUG-024 · Rate-Limit Retries Ignored Backoff Delays

Field	Detail
Issue	Transient Groq rate-limit failures (HTTP 429) retried immediately without respecting exponential backoff or provider retry guidance.
Found Date	2026-06-10 (observed during high-volume ladder and dry-run execution)
Status	Resolved
Severity	HIGH
File	llm_caller.py
Description	Extended dry-run execution triggered Groq token-per-day rate limits across retrieval validation, merge validation, chunk merging, and agent retrieval phases. Retries occurred immediately after failure, causing repeated 429 responses and unnecessary API pressure. In some cases, repeated retries exhausted the retry budget without allowing sufficient recovery time.
Root Cause	Retry logic handled transient failures but did not properly implement exponential backoff behavior or honor provider-supplied retry-after guidance.
Solution	Implemented exponential backoff with jitter and support for retry-after headers. Added dedicated test coverage validating retry-after handling, exponential delay growth, retry exhaustion, and successful recovery after transient rate-limit failures.
Date Resolved	2026-06-10

BUG-025 · Configuration Flags Could Not Truly Disable Output-Repair Paths

Field	Detail
Issue	Several stage-specific output-repair flags existed in config.py, but disabling them exposed execution paths that had never been exercised independently.
Found Date	2026-06-10 (during run_combinations.py feature-combination testing)
Status	Resolved
Severity	HIGH
File	validators.py, context_compression.py, self_learner.py
Description	The new configuration architecture introduced independent output-repair switches for retrieval validation, merge validation, compression validation, QA generation, and global repair. Dry-run combinations revealed that many code paths only functioned correctly when output repair remained enabled. Turning off repair exposed hidden dependencies and assumptions throughout the validation and compression layers. As a result, several feature combinations crashed or returned invalid execution states despite being considered supported configurations.
Root Cause	The architecture was originally built around mandatory fix_llm_output() usage. The later addition of feature flags created configuration states that had never been tested independently.
Solution	Audited all structured-output callsites and ensured that disabling repair still routes responses through a safe parsing layer before downstream consumption. Added feature-combination testing to validate both repaired and non-repaired execution paths.
Date Resolved	2026-06-10

BUG-026 · Feature Combination Testing Exposed Hidden Pipeline Coupling

Field	Detail
Issue	Multiple supposedly independent pipeline stages contained implicit dependencies on upstream stages being enabled.
Found Date	2026-06-10 (during randomized configuration testing)
Status	Resolved
Severity	MEDIUM
File	Multiple pipeline modules
Description	run_combinations.py generated feature combinations that disabled various retrieval, compression, validation, and learning stages. Several combinations failed despite no direct dependency being documented. Components assumed that earlier stages had already normalized, validated, compressed, or repaired data before execution. This created unexpected failures when intermediate stages were bypassed.
Root Cause	Architectural assumptions accumulated organically as features were added. Modules consumed data in forms guaranteed by earlier stages without independently validating their own inputs.
Solution	Added defensive handling throughout the pipeline and updated fallback paths to operate correctly regardless of which optional stages are enabled. Configuration combinations are now treated as first-class supported execution modes rather than edge cases.
Date Resolved	2026-06-10

BUG-027 · Dry Runs Produced Non-Isolated Experimental Results

Field	Detail
Issue	Repeated ladder and configuration-combination runs could influence later runs through shared system state.
Found Date	2026-06-10 (during repeated dry-run execution)
Status	Resolved
Severity	MEDIUM
File	Testing infrastructure and execution workflow
Description	While validating multiple configuration combinations, it became apparent that some pipeline behavior could be affected by state accumulated during earlier runs. This reduced confidence when comparing feature combinations because changes in observed results could originate from previous execution history rather than the configuration currently under test.
Root Cause	Experimental execution assumed isolation between runs while some system components persisted state across executions.
Solution	Updated testing workflow and configuration handling to better isolate experimental runs and ensure that feature-combination comparisons reflect only the currently active configuration.
Date Resolved	2026-06-10

BUG-028 · Missing Test Coverage for Configuration-Specific Execution Paths

Field	Detail
Issue	Existing tests validated the primary execution path but did not exercise alternate paths created by configuration switches.
Found Date	2026-06-10 (during dry-run analysis)
Status	Resolved

Severity	MEDIUM
File	Project-wide testing strategy
Description	The introduction of a large number of feature toggles significantly increased the number of possible execution paths. Existing testing focused on the fully enabled pipeline and therefore failed to detect issues that only appeared when specific stages were disabled. Several bugs discovered today existed solely because these alternate execution paths had never been exercised.
Root Cause	Test coverage evolved around the default production configuration rather than around the full configuration space.
Solution	Introduced configuration-combination testing through <code>run_combinations.py</code> and used it as a systematic mechanism for validating alternative execution paths.
Date Resolved	2026-06-10

BUG-029 · Final Result/Feedback Returned Last Retrieval Only, Not Accumulated Per-Track State

Field	Detail
Issue	The returned answer's evidence and the persisted feedback record contained only the chunks from the most recent <code>retrieve_documents</code> call, not the accumulated per-track chunks that supported the answer.
Found Date	2026-06-11 (first post-separation dry run)
Status	Closed
Severity	HIGH
File	<code>agent_query.py</code> , <code>feedback_store.py</code>
Description	The dry-run log showed the answer drafted from <code>`documents=6</code>
Root Cause	<code>last_retrieval_fields()</code> consumed the retriever's per-call last-state getters instead of the orchestrator's

	<code>agent_state["accumulated_document_chunks"]</code> and <code>agent_state["accumulated_learned_qa_chunks"]</code> . The result and feedback shapes therefore reflected a single retrieval call, not the post-validation/compression state that actually informed the answer.
Solution	Replaced <code>last_retrieval_fields()</code> with an accumulated-state projection that reads both tracks from <code>agent_state</code> , strips embeddings, and normalises each chunk to <code>{content, source}</code> . Wired the projection into every successful and failed result path. Updated <code>FeedbackStore.log()</code> to accept the normalised shape and split legacy single-list <code>chunks</code> records by <code>source</code> at read time for backward compatibility. Added regression coverage in <code>test_retrieval_separation.py</code> .
Date Resolved	2026-06-11

BUG-030 · learned_qa Collection Persisted with L2 Distance While documents Uses Cosine

Field	Detail
Issue	The live <code>learned_qa</code> Chroma collection was using L2 distance while <code>documents</code> was using cosine, but both were ranked by <code>similarity_score = 1 - dist</code> and filtered through the same <code>MIN_SIMILARITY = 0.5</code> threshold.
Found Date	2026-06-11 (first post-separation dry run)
Status	Closed
Severity	HIGH
File	<code>agent_query.py</code> , <code>self_learner.py</code> , <code>retriever.py</code> , new <code>learned_qa_store.py</code>
Description	Runtime collection inspection showed <code>documents</code> configured with <code>hnsw:space = cosine</code> and <code>learned_qa</code> configured with <code>hnsw:space = l2</code> . The dry-run log showed relevant learned-QA results barely surviving the threshold around 0.523–0.533, while other queries returned zero learned-QA results despite the collection containing semantically relevant entries. <code>retriever.py</code> computed <code>similarity_score = 1 - distance</code> for both collections,

	which is correct for cosine on unit-normalised vectors but produces $2 \cdot \cos(\theta) - 1$ for L2 (Research topic 27). Therefore learned-QA chunks were being ranked and threshold-filtered under incompatible score semantics, potentially dropping high-priority learned answers before the ADR-030 precedence rule could even apply. Root cause was a startup ordering bug: <code>agent_query.py</code> created the collection without <code>hnsw:space</code>, defaulting to L2; <code>self_learner.py</code> later requested cosine on the same name, but <code>get_or_create_collection()</code> does not migrate distance metric — the HNSW index has to be rebuilt.
Root Cause	<code>learned_qa</code> was created without specifying <code>hnsw:space</code>, and Chroma's default is L2. The <code>self_learner.py</code> request for cosine after the fact had no effect because metadata changes do not rebuild the HNSW index. The system had no single canonical creation point that enforced cosine.
Solution	Added <code>learned_qa_store.py</code> providing <code>get_or_create_learned_qa_collection()</code> — a single factory that guarantees cosine, with a snapshot/restore/verify/rollback migration path for existing L2 collections (see ADR-031). Wired the factory into both <code>agent_query.py</code> startup and <code>self_learner.py</code> so, the two code paths share one canonical collection handle. Live production migration completed successfully: 374 L2 entries → 374 cosine entries, all IDs preserved. Regression test added in <code>test_retrieval_separation.py</code> covering the L2→cosine migration.
Date Resolved	2026-06-11

BUG-031 · Quality Judge Loses Document Track Due to 3000-Char Combined-Context Truncation

Field	Detail
Issue	<code>check_answer_quality</code> is called with <code>context[:3000]</code>. Because the precedence-aware combined context writes the learned-QA section first, a large learned-QA block can push the entire document section out of the truncation window — so the judge approves an answer without ever seeing the document chunks that supported its document-grounded claims.
Found Date	2026-06-11 (second post-separation dry run)
Status	Open

Severity	HIGH
File	<code>tools.py</code> (judge callsite), <code>context_compression.py</code> (<code>format_precedence_context_for_llm</code>)
Description	Dry-run lines 576–590 show a query that produced <code>documents=6</code>, <code>learned_qa=7</code> and a 6,478-character combined context. <code>format_precedence_context_for_llm()</code> emits <code>[CONFLICT RESOLUTION RULE] → [LEARNED QA CONTEXT - HIGH PRIORITY] → [DOCUMENT CONTEXT - SECONDARY]</code> in that fixed order. The judge then sliced the first 3,000 chars and returned <code>OK</code>. In this case the learned-QA section alone exceeded 3,000 chars, so the document section was entirely outside the slice. The draft answer contained document-supported claims; the judge approved them without ever seeing the supporting evidence. This is the inverse of the failure mode that the precedence ordering was designed to prevent: precedence ordering correctly steers the answer, but the judge sees only the head of the combined string.
Root Cause	The judge call uses a single combined context with a single character cap, applied to a string whose two tracks are concatenated in a fixed order. Truncating a structured two-section input as if it were undifferentiated text discards one section whenever the first section exceeds the cap.
Solution	Format two independently-bounded sections for the judge — give each track its own character/token allowance and include both, rather than truncating the head of the combined string. The simplest implementation is a <pre>format_precedence_context_for_judge(learned_qa_chunks, document_chunks, per_track_budget)</pre> helper that produces the same labelled-section layout but with per-section caps. The judge prompt may also need a small update so it knows it may be receiving an excerpt of each section rather than the full context.
Date Resolved	—

BUG-032 · Final-Answer Generation Bypasses Grounding and Conflict-Precedence Validation

Field	Detail
Issue	The DRAFT answer is validated by <code>check_answer_quality</code> , but a subsequent final-answer generation step runs after the judge and the resulting string is returned directly without any further validation for grounding or learned-QA precedence.
Found Date	2026-06-11 (second post-separation dry run)
Status	Open
Severity	HIGH
File	<code>agent_query.py</code>
Description	The dry-run log at lines 602–606 shows the draft answer judged <code>OK</code> , then a new final-answer generation runs at lines 612–620 producing the string returned to the user. No validator runs between the final generation and the return path. The final generation can therefore alter the approved draft, introduce unsupported claims, drop the learned-QA precedence behaviour, or pick a conflicting document claim — and no part of the pipeline detects it. ADR-030's conflict-resolution rule is enforced as a prompt instruction in the final-answer prompt, so the precedence behaviour is best-effort rather than verified. This is structurally the same risk as a CAQ that judges a draft and then accepts a regenerated answer without re-judging.
Root Cause	The pipeline contains two answer-producing steps (DRAFT and final) but only one quality gate, sitting between the two. The final answer is treated as a formatting/cleanup pass rather than as a generation step that could diverge from the approved draft.
Solution	Two options. (a) Run a final-answer validator with an explicit conflict-precedence criterion — pass it the final answer, the per-track contexts, and a check that the final answer's claims are either grounded in the documents or supported by (or at least not contradicted by) the learned-QA track. (b) Drop the post-judge final-answer generation entirely and return the already-approved draft directly — cheaper and removes the divergence risk, at the cost of forfeiting any formatting/prose-cleanup the final step was performing. (b) is the smaller change and is the recommended

	starting point; if its output quality is acceptable, (a) is unnecessary.
Date Resolved	—

BUG-033 · Thumbdown Variants Store Pre-Validation Chunks

Field	Detail
Issue	<code>failed_variants.json</code> and the chunk snapshots inside <code>user_thumbdowns.json</code> are captured from the raw retrieval output, before per-track relevance validation drops irrelevant chunks. Replay logic therefore treats rejected chunks as if they had supported the failed answer.
Found Date	2026-06-11 (second post-separation dry run)
Status	Open
Severity	MEDIUM
File	<code>agent_query.py</code>
Description	For the first query in the dry run, raw retrieval produced <code>learned_qa=5, documents=3</code> , but <code>validate_track()</code> kept only <code>learned_qa=3/5, documents=2/3</code> . Variant capture happens at the retrieval callsite (before <code>validate_track</code>), so the variant record stores 5+3 chunks where the actual answer was built from 3+2. If the user later thumbs-down this answer, the avoid-prior-failure injection will list two chunks that the validator already rejected as not supporting the answer in the first place. The downstream effect is that the model is steered away from retrieval directions it had already correctly rejected, which can both waste retrieval budget on diversification away from non-issues and miss the actual failure direction.
Root Cause	Variant capture is positioned at the raw retrieval boundary rather than after validation. The original single-stream architecture captured "what was retrieved" as the unit of evidence; the new two-track architecture needs to capture "what survived validation per track" as the unit of evidence, but the capture site was not moved.
Solution	Either (a) move variant capture to after <code>validate_track()</code> so the

	recorded chunks are validation-survivors, or (b) record both <code>raw_*</code> and <code>validated_*</code> fields per variant and have the avoid-prior-failure injection consume <code>validated_*</code> . (a) is the minimal change and matches the answer's actual evidence.
Date Resolved	—

BUG-034 · "Sources Searched" Preview Hides Document Retrieval

Field	Detail
Issue	Per-variant "sources searched" previews are taken as <code>result[:200]</code> of a string whose learned-QA section is written first. The preview therefore always shows the head of the learned-QA section and never reveals that any documents were retrieved.
Found Date	2026-06-11 (second post-separation dry run)
Status	Open
Severity	MEDIUM
File	<code>agent_query.py</code> (sources/preview construction), <code>tools.py</code> (tool-result string ordering)
Description	Dry-run lines 625–637 show every source preview beginning with <code>[LEARNED QA RESULTS - HIGH PRIORITY]</code> , even for variants where the final answer explicitly cited document sources. The tool-result string is intentionally ordered learned-QA-first to support the precedence rule (ADR-030), but the preview slice <code>result[:200]</code> then reflects only that ordering decision, not the actual retrieval distribution. Result: feedback summaries, console output, and audit displays all under-report document retrieval and can give the false impression that the system is operating learned-QA-only.
Root Cause	The preview is derived from the head of a single concatenated string instead of from the structured per-track data. The ordering rule (precedence) and the preview rule (representativeness) have incompatible requirements when the preview is a head slice.
Solution	Replace the single <code>preview</code> field with structured <code>learned_qa_preview</code> and <code>document_preview</code> fields, each derived from its own track and bounded independently. Console/audit

	output reads both. The combined string remains learned-QA-first for the LLM, but never feeds the preview.
Date Resolved	—

BUG-035 · Retrieval Validator Returns PASS Even When per_chunk Counts Are a Minority

Field	Detail
Issue	The retrieval-validation LLM returns a top-level verdict: PASS while its own per_chunk array shows a minority of chunks relevant (e.g. 2/5). The orchestrator trusts the top-level field directly, so the agent never receives the PARTIAL/FAIL retry guidance it should have received.
Found Date	2026-06-11 (second post-separation dry run)
Status	Open
Severity	MEDIUM
File	prompts.py (RETRIEVAL_JUDGE_PROMPT), validators.py (validate_retrieval), agent_query.py (verdict consumption)
Description	Dry-run line 314 reports verdict=PASS relevant=2/5. The retrieval judge prompt defines PASS as requiring a majority of chunks to be relevant, but the LLM emitted PASS while its own per-chunk verdicts show 2/5 — a minority. The orchestrator consumes the top-level verdict field rather than deriving it from the per-chunk counts, so the FAIL/PARTIAL retry guidance never fires, and the agent does not reformulate. The same risk applies to the inverse case (LLM emits FAIL while per_chunk shows majority relevant). Trusting an LLM-emitted summary field that disagrees with its own structured evidence is a general anti-pattern.
Root Cause	The validator's overall verdict is consumed as-emitted instead of being computed from the verifiable per_chunk array. There is no internal consistency check between the two.
Solution	Derive the overall verdict deterministically from per_chunk after

	the LLM call returns and the schema is repaired by <code>fix_llm_output</code>. Treat the LLM's top-level verdict as advisory only — log a warning if it disagrees with the computed verdict. A reasonable rule: <code>PASS</code> when $\geq \lceil n/2 \rceil$ chunks are relevant, <code>PARTIAL</code> when $1 \leq \text{relevant} < \lceil n/2 \rceil$, <code>FAIL</code> when 0 are relevant. Same fix lets the orchestrator's <code>PARTIAL/FAIL</code> retry path operate on a stable signal.
Date Resolved	—

BUG-036 · Legacy-Format Splitter Misclassifies Merged Learned-QA Chunks

Field	Detail
Issue	When <code>merge_similar_chunks()</code> merges multiple learned-QA chunks, it joins their source fields into <code>"learned_qa, learned_qa, ..."</code> . The legacy feedback splitter, which matches only the exact string <code>source == "learned_qa"</code> , then misclassifies these merged chunks as document chunks.
Found Date	2026-06-11 (second post-separation dry run)
Status	Open
Severity	MEDIUM
File	<code>context_compression.py</code> (<code>_merge_similar_chunks</code> source field construction), <code>feedback_store.py</code> (legacy splitter at line ~39)
Description	The merge implementation in <code>context_compression.py</code> produces chunks whose <code>metadata.source</code> is the comma-joined list of input sources, e.g. <code>"learned_qa, learned_qa"</code> or <code>"learned_qa, learned_qa, learned_qa"</code>. The legacy splitter in <code>feedback_store.py</code> — which exists to convert old single-list <code>chunks</code> records into the new per-track shape at read time — only routes chunks to the learned-QA list when <code>source == "learned_qa"</code> exactly. A merged chunk with <code>source = "learned_qa, learned_qa"</code> does not match and falls through into the documents track. The same issue affects merged document chunks whose sources concatenate (e.g. <code>"file_a.pdf, file_b.pdf"</code>), but mis-routing within the documents track is less consequential than mis-routing across tracks.

Root Cause	The chunk-merge step loses the track (provenance bucket) by encoding it into a free-form joined string field that was originally meant only for document filename. The legacy splitter then has to reverse-engineer track membership from that string, with a substring match that misses every multi-source case.
Solution	Preserve an explicit <code>track</code> field (e.g. <code>"documents"</code> or <code>"learned_qa"</code>) on every chunk through retrieval, validation, merge, and compression. Make the legacy splitter route by <code>track</code> when present, and fall back to the exact <code>source == "learned_qa"</code> rule only when <code>track</code> is absent. The merge step then no longer must encode provenance into the source string; the source string can keep its filename-list semantics for documents while learned-QA chunks carry their track explicitly.
Date Resolved	—

BUG-037 · Validator Repair Logging Prints "Failed" On Success

Field	Detail
Issue	The validator-side log message emitted around <code>fix_llm_output</code> repair calls prints <code>failed to fix malformed LLM output</code> even when the repair pass succeeds, and a valid verdict is produced.
Found Date	2026-06-11 (second post-separation dry run)
Status	Closed
Severity	LOW
File	<code>app_workflow/services/validators.py</code> (<code>validate_retrieval()</code>)
Description	Dry-run logs repeatedly show the line <code>failed to fix malformed LLM output</code> immediately followed by a successfully parsed verdict being consumed by downstream code. The log message is on the wrong branch — the success path is printing the failure-path text. The functional behaviour is correct; only the diagnostics are misleading. The cost is real: ladder runs, dry-run reviews, and debugging sessions all rely on log messages to triage failures, and a repair layer that prints "failed" on every success makes its actual failure mode (when repair really fails) impossible to spot.

Root Cause	Inverted (or mistyped) log message on the success branch of the validator's repair-handling block.
Solution	Swap the log message on the <code>ok is True</code> branch to indicate successful repair (or remove the line entirely on success). Keep the failure-path message exclusively on the <code>ok is False</code> branch. One-line fix. Resolved 2026-07-01: fixed in <code>validate_retrieval()</code> — moved the <code>WARNING: failed to fix</code> message into the actual failure branch and added <code>INFO: successfully fixed</code> on the success branch. Investigation at fix time found the bug was isolated to <code>validate_retrieval()</code>; <code>validate_merge</code>, <code>validate_redundancy</code>, and <code>validate_lbc</code> were already logging correctly, so this was narrower than originally filed.
Date Resolved	2026-07-01

BUG-038 · Mechanical print→logger Migration Dropped Debug print/write Blocks → Empty Diagnostic Files

Field	Detail
Issue	The initial <code>print()</code> → <code>logger.<level>()</code> conversion pass during the 2026-06-15 logging refactor mechanically replaced single-line <code>print()</code> calls but removed (rather than converted) several multi-line <code>print()</code> and <code>f.write(...)</code> debug blocks. After the migration, the diagnostic output files (<code>agent_query.debug.log</code> , <code>agent_query.error.log</code> , <code>llm_data_check.txt</code> , <code>llm_json_tries.txt</code>) were all created on startup but stayed completely empty across complete agent runs.
Found Date	2026-06-15
Status	Closed
Severity	HIGH
File	<code>fix_llm_output.py</code> , <code>agent_query.py</code> , <code>logger_config.py</code>
Description	The mechanical migration covered ~370 single-line <code>print(...)</code> calls successfully but missed two categories of pre-migration output: (a) the multi-line <code>with open("run_logs/llm_data_check.txt", "a") as f: f.write(...)</code> and

	... llm_json_tries.txt..., f.write(...) blocks in fix_llm_output.py that produced the Verify_And_Correct and LLM_Json_Repair per-call payload traces; and (b) several detailed print(...) blocks that had been collateral-deleted during the cleanup pass rather than rewritten. The result was that the new file handlers in logger_config.py opened the target files (creating them with zero bytes) but no module ever emitted a record into them, so they remained empty after a successful end-to-end run. The detailed debug content (LLM input/output, retrieved chunks, draft preview, system-prompt assembly traces, Verify_And_Correct payloads, LLM_Json_Repair attempts) had simply been deleted from the codebase.
Root Cause	The migration regex matched only inline print(...) patterns and did not handle multi-line with open() as f: f.write(...) blocks. The cleanup pass that followed the regex conversion was performed manually and deleted several debug blocks that the operator interpreted as redundant rather than translating them to logger.debug(...). There was no automated check that the new logger handlers were actually receiving records before the migration was declared complete.
Solution	Restored the full Verify_And_Correct payload trace (raw input, schema, repaired output, correction-applied flag) and the full LLM_Json_Repair attempt trace (raw input, repair prompt, attempts, final outcome) through dedicated llm_data_check_logger.debug(...) and llm_json_tries_logger.debug(...) calls using logging.getLogger("llm_data_check") and logging.getLogger("llm_json_tries"). Set propagate=True on both diagnostic loggers in logger_config.py so their records reach the root logger and through it the timestamped debug file. Verified by re-running agent_query.py and confirming the per-run debug file (agent_query_20260615_171551.debug.log) contains the expected full DEBUG trace. The agent_query.error.log and old agent_query.debug.log files (from the brief dual-file design before the timestamped-per-run pattern was adopted) are now stale and can be deleted.
Date Resolved	2026-06-15

BUG-039 · Severity-Level Mismatches In context_compression.py And self_learner.py After Print→Logger Conversion

Field	Detail
Issue	The mechanical <code>print()</code> → <code>logger.<level>()</code> conversion pass assigned uniform default levels (mostly <code>debug</code> or <code>warning</code>) that did not match the semantic meaning of several messages. Non-retryable errors logged at <code>warning</code> ; "gave up on a chunk after retries" logged at <code>debug</code> ; messages already prefixed <code>[ERROR]</code> in their text logged at <code>debug</code> or <code>warning</code> .
Found Date	2026-06-15
Status	Closed
Severity	MEDIUM
File	<code>context_compression.py</code> , <code>self_learner.py</code>
Description	An audit pass over the converted code found six callsites where the assigned level was semantically wrong. In <code>context_compression.py</code>: the three "non-retryable error" branches in <code>CHUNK MERGE</code>, <code>DC</code>, and <code>LBC</code> (each representing a stage giving up after a fatal LLM call) were emitting at <code>warning</code> rather than <code>error</code>, and the <code>LBC</code> "parse failed after N attempts" branch (representing the function giving up on a chunk after retry exhaustion) was emitting at <code>debug</code>, making it invisible on the console at the new <code>INFO</code> threshold. In <code>self_learner.py</code>: three callsites whose message text already begins with <code>[ERROR]</code> (the <code>LLM-distillation-failed</code> branch and two <code>QA-pair-parse-failed</code> branches) were emitting at <code>debug</code> or <code>warning</code>, so the prefix and the level disagreed. The functional behaviour was correct — the failures still fell through to the right error path — but the diagnostics were inconsistent with the level filtering the rest of the codebase had been organised around, and would have hidden real failures from the console at <code>INFO</code>.
Root Cause	The conversion script applied a default level (<code>warning</code> for messages that looked like fallbacks, <code>debug</code> for everything else) without inspecting the surrounding control flow or the message text. The audit step in the migration plan was supposed to catch this but was skipped on the first pass.
Solution	Promoted three <code>context_compression.py</code> messages (<code>CHUNK</code>

	MERGE / DC / LBC non-retryable error) from <code>warning</code> to <code>error</code> . Promoted the LBC "parse failed after N attempts" message from <code>debug</code> to <code>warning</code> . Promoted three <code>self_learner.py</code> [ERROR]-prefixed messages from <code>debug/warning</code> to <code>error</code> . No prompt or control-flow changes were required. Audit-pass discipline added to the migration checklist: after a mechanical level assignment, every <code>error/warning/debug</code> call-site must be re-read against the surrounding code path and the message text.
Date Resolved	2026-06-15

BUG-040 · DC Stage Shape Mismatch When ENABLE_COMPRESSION_VALIDATION=False

Field	Detail
Issue	When <code>ENABLE_COMPRESSION_VALIDATION</code> is <code>False</code> , the DC stage's <code>valid_groups</code> is <code>list[list[dict]]</code> (each entry is the members list directly), but the consumption loop always expects <code>list[{"members": [...]}]</code> (each entry is a dict with a "members" key). The loop raises <code>TypeError</code> on the <code>False</code> branch.
Found Date	2026-06-17 (surfaced while expanding <code>tempFile.py</code> to call <code>context_compression</code> functions directly)
Status	Closed
Severity	MEDIUM
File	<code>app/context_compression.py</code> (DC consumption loop, ~line 401)
Description	When <code>ENABLE_COMPRESSION_VALIDATION=True</code> , <code>confirmed_groups</code> comes from <code>validate_redundancy()</code> as <code>list[RedundancyGroupResult]</code> — each entry is a dict with a "members" key. When <code>ENABLE_COMPRESSION_VALIDATION=False</code> , <code>confirmed_groups = valid_groups</code> which is <code>list[list[dict]]</code> — each entry is the raw members list. The consumption loop at the merge step always does <code>group["members"]</code> , so the <code>False</code> branch breaks as soon as more than zero groups are confirmed. This was invisible in <code>app/agent_query.py</code> during recent runs because <code>ENABLE_DC_COMPRESSION</code> and <code>ENABLE_COMPRESSION_VALIDATION</code> were toggled together in the same ladder step; it only became observable when

	<code>tempFile.py</code> called the function with validation explicitly disabled.
Root Cause	The two branches of the <code>confirmed_groups</code> assignment produce different shapes, and the consumption loop was only written for the validated-output shape.
Solution	One-line fix: when <code>ENABLE_COMPRESSION_VALIDATION=False</code> , wrap <code>valid_groups</code> items as <code>[{"members": group} for group in valid_groups]</code> before the consumption loop, so both branches produce the same <code>list[{"members": [...]}]</code> shape.
Date Resolved	2026-06-17

BUG-041 · `chunk_seq` Not Promoted From metadata In `tempFile.py` Retrieve Node → NAC Skips Neighbor Detection

Field	Detail
Issue	NAC checks <code>c.get("chunk_seq")</code> at the top level of each chunk dict. The raw retriever output stores <code>chunk_seq</code> at <code>c["metadata"]["chunk_seq"]</code> . When the retrieve node passed raw retriever output directly to <code>compress_neighbor_chunks</code> , NAC never found <code>chunk_seq</code> and therefore could not perform any neighbor-aware merge.
Found Date	2026-06-17 (surfaced while building <code>tempFile.py</code>)
Status	Closed
Severity	LOW (only affected the sandbox <code>tempFile.py</code> ; <code>app/agent_query.py</code> was never affected — <code>accumulate_track()</code> already promoted <code>chunk_seq</code> to the top level before calling compression)
File	<code>tempFile.py</code> retrieve node (now deleted); root cause is a documentation/convention gap — the required chunk shape for compression callers is not formally specified anywhere
Description	<code>app/agent_query.py</code> 's <code>accumulate_track()</code> helper explicitly promotes <code>chunk_seq</code> from metadata to the top of the chunk dict when building <code>accumulated_document_chunks</code> . This normalisation step is not part of <code>retriever.retrieve()</code> or

	<code>retriever.retrieve_separate()</code> — it is the caller's responsibility. <code>tempFile.py</code>'s <code>retrieve</code> node passed the raw <code>retriever.retrieve()</code> output straight into <code>compress_neighbor_chunks</code>, skipping the normalisation. The symptom was that NAC ran without error but performed zero merges (no chunks had <code>chunk_seq</code>, so no neighbors were identified).
Root Cause	The chunk-shape contract required by compression callers (<code>chunk_seq</code> at the top level) is implicit — it lives only in <code>accumulate_track()</code> in <code>agent_query.py</code> and is not documented or enforced at the retriever API boundary.
Solution	In <code>tempFile.py</code>'s <code>retrieve</code> node, flatten each retrieved doc to the shape compression expects: <code>{"content": ..., "source": ..., "chunk_seq": c["metadata"].get("chunk_seq", 0), ...}</code> before storing in state. For <code>app_workflow/</code>, the <code>retrieve.py</code> node applies the same flattening so <code>validate_retrieval.py</code> and downstream compression nodes always receive normalised chunks.
Date Resolved	2026-06-17

BUG-042 · `httpx.HTTPStatusError (429)` and `httpx.ConnectError` Not Caught by `llm_caller.py` in `app_workflow/`

Field	Detail
Issue	Two HTTP error types surfaced in <code>app_workflow/</code> 's debug log that were not caught by the existing <code>llm_caller.py</code> error handlers and therefore propagated as unhandled exceptions, crashing the LangGraph node.
Found Date	2026-06-18 (observed in <code>rag_langgraph_20260618_183140.debug.log</code>)
Status	Closed
Severity	HIGH
File	<code>app_workflow/services/llm_caller.py</code>
Description	The existing handlers in <code>llm_caller.py</code> caught <code>groq.APIStatusError</code> (for Groq-SDK-wrapped 429s) and <code>groq.APIConnectionError</code> (for Groq-SDK-wrapped connection failures). In some cases the Groq SDK's internal retry loop

	exhausted its own retries and re-raised the underlying <code>httpx.HTTPStatusError</code> (with status 429) and <code>httpcore.ConnectError</code> (DNS resolution failure, <code>[Errno 11001]</code> <code>getaddrinfo</code> failed) directly, bypassing the SDK wrappers. These fell through to the generic <code>Exception</code> catch-all which logged them with <code>logger.exception</code> but did not map them to a structured <code>LLMErrorKind</code> — causing the caller to receive an unstructured error path and, in the LangGraph context, crash the node.
Root Cause	<code>httpx.HTTPStatusError</code> and <code>httpx.ConnectError</code> are lower-level than the Groq SDK exception types and are only raised when the SDK's own retry logic is exhausted. They were not included in the original <code>llm_caller.py</code> exception hierarchy because they were not observed in early dry runs.
Solution	Added two <code>except</code> blocks before the generic <code>Exception</code> catch-all in <code>app_workflow/services/llm_caller.py</code> : <code>except httpx.HTTPStatusError as e</code> — maps <code>e.response.status_code == 429</code> → <code>LLMErrorKind.RATE_LIMIT</code> (so the existing retry loop backs off and retries), <code>5xx</code> → <code>LLMErrorKind.SERVER_ERROR</code> , other → <code>LLMErrorKind.UNKNOWN</code> ; <code>except httpx.ConnectError</code> — maps to <code>LLMErrorKind.CONNECTION</code> , same as <code>groq.APIConnectionError</code> . Added <code>import httpx</code> at the top of the file.
Date Resolved	2026-06-18

BUG-043 · app_workflow/ Does Not Write failed_variants.json After Each Run

Field	Detail
Issue	<code>app_workflow/nodes/user_input.py</code> reads <code>failed_variants.json</code> on startup (to pre-populate <code>blocked_variants</code> in state) but no node ever writes newly discovered failing variants back to the file. In <code>app/agent_query.py</code> , this write happened after each <code>run_agent()</code> call.
Found Date	2026-06-18
Status	Closed
Severity	MEDIUM

File	app_workflow/ — missing write-back; compare app/agent_query.py lines ~1200–1201
Description	After a run completes (whether via generate_answer or no_context_answer), the workflow accumulates a set of query variants that were tried and produced no useful context. In app/, these are saved to data/feedback/failed_variants.json so that future runs can pre-block those trajectories. app_workflow/'s graph has no node that performs this save. The consequence is that the thumbsdown-blocking mechanism works within a session (variants are in state) but does not persist across sessions — the same failing trajectories can be attempted again on the next process start.
Root Cause	The write-back was not ported during the June 17–18 build. It requires access to the variants tried during the run and a call to feedback_store.save_failed_variants() (or equivalent) after generate_answer/no_context_answer complete.
Solution	Added load_failed_variants() / save_failed_variants() to services/feedback_store.py. retrieve.py now emits newly_failed_variants: Annotated[list[str], operator.add] (zero-chunk variants, accumulated across all parallel Send branches) into state. generate_answer.py and no_context_answer.py both call feedback_store.save_failed_variants() at the end of their run.
Date Resolved	2026-06-19

BUG-044 · app_workflow/ Thumbsdown Records Written With Empty variants Field

Field	Detail
Issue	cmd_bad in app_workflow/nodes/commands.py calls feedback_store.mark_last_bad(feedback=...) but does not pass a variants argument. Thumbsdown entries are written to user_thumbsdowns.json with an empty variants array, so the future blocked-variant injection has nothing to block.
Found Date	2026-06-18
Status	Closed

Severity	MEDIUM
File	app_workflow/nodes/commands.py (cmd_bad node)
Description	In app/agent_query.py line ~1117, mark_last_bad() is called with variants=last_result.get("variants", []) — the list of query variants that were tried in the last run. app_workflow/s cmd_bad node has access to state, but the variants tried in the most recent run are not currently stored in a state field accessible after the fact. The query_variants field in state holds the variants generated by generate_query_variants for the current invocation, which is the correct source, but cmd_bad runs in a separate graph invocation (the user types bad as a new input) — by the time cmd_bad runs, the previous run's state is gone.
Root Cause	The workflow does not persist the last run's query_variants between invocations. cmd_bad runs in its own graph invocation and cannot see the prior invocation's state without an explicit persistence mechanism.
Solution	retrieve.py now emits variants_with_chunks: Annotated[list[dict], operator.add] (every variant's retrieved chunks, both tracks) into state. generate_answer.py and no_context_answer.py cache state["variants_with_chunks"] into a new modulelevel services.last_variants_with_chunks sidecar at the end of each run. cmd_bad now reads services.last_variants_with_chunks and passes it as variants= to mark_last_bad(), the same sidecar pattern app/agent_query.py's REPL loop uses for last_result.
Date Resolved	2026-06-19

BUG-045 · Bare Intra-Package Imports in app_workflow/services/ Break on Startup (ModuleNotFoundError: embedding_manager)

Field	Detail
Issue	services/services.py, services/retriever.py, services/validators.py, services/self_learner.py, and services/fix_llm_output.py imported sibling modules with bare names (from embedding_manager import ...) instead of relative imports, raising ModuleNotFoundError: No module named 'embedding_manager' the moment app_workflow/main.py was run
Found Date	2026-06-19
Status	Closed
Severity	HIGH
File	app_workflow/services/services.py, retriever.py, validators.py, self_learner.py, fix_llm_output.py
Description	services/ is a Python package, so its modules can only resolve sibling imports as from .embedding_manager import ... (relative) or via the fully-qualified app_workflow.services.embedding_manager path. The bare-name imports inherited from app/'s flat-module convention only worked there because app/ modules are not packaged — they're imported with app/ itself on sys.path. The June 17–18 port carried the bare-import style over unchanged.
Root Cause	app/'s flat module layout (no __init__.py, modules imported with the directory on sys.path) and app_workflow/services/'s package layout (services/ has package semantics once app_workflow.services.services is imported as a dotted path) require different import styles; the port copied the source files without adjusting imports for the new layout.
Solution	Converted all five files' intra-services/ imports to relative form (from .embedding_manager import EmbeddingManager, etc.).
Date Resolved	2026-06-19

**BUG-046 · app_workflow/main.py Run from Inside app_workflow/
Cannot Resolve from app_workflow.config import ...
(ModuleNotFoundError: app_workflow)**

Field	Detail
Issue	After fixing BUG-045, a second <code>ModuleNotFoundError: No module named 'app_workflow'</code> surfaced from <code>services/retriever.py</code> 's <code>from app_workflow.config import RETRIEVAL_TOP_K, RETRIEVAL_TOP_L, MIN_SIMILARITY</code>
Found Date	2026-06-19
Status	Closed
Severity	HIGH
File	<code>app_workflow/main.py</code>
Description	<code>main.py</code> is launched with the working directory set to <code>app_workflow/</code> (<code>python .\main.py</code> from inside that folder), so <code>app_workflow/</code> itself is on <code>sys.path</code> — which lets bare imports like <code>from graph import build_graph</code> resolve, but does not put the project root (<code>RAG-work/</code>) on <code>sys.path</code> . Several <code>services/</code> modules need the project root on the path to resolve <code>from app_workflow.config import ...</code> (a dotted, fully-qualified import used throughout <code>services/</code> and <code>nodes/</code>).
Root Cause	Two different import conventions are in play simultaneously: bare imports that assume <code>app_workflow/</code> is the working directory, and dotted imports that assume the project root is on <code>sys.path</code> . Only the first was satisfied by the normal <code>python .\main.py</code> invocation.
Solution	Inserted the project root (<code>RAG-work/</code> , derived from <code>file</code>) at the front of <code>sys.path</code> at the very top of <code>main.py</code> , before any other imports execute. <code>app_workflow/</code> (the CWD) remains on the path too, so the existing bare imports inside <code>main.py</code> continue to work alongside the now-resolvable dotted imports.
Date Resolved	2026-06-19

BUG-047 · FAILED_VARIANTS_PATH Resolves Outside the Project (Extra ../ in app_workflow/config.py)

Field	Detail
Issue	FAILED_VARIANTS_PATH in app_workflow/config.py was built with one extra ../ relative to the other data/feedback/ path constants, resolving outside the project root instead of to data/feedback/failed_variants.json
Found Date	2026-06-19
Status	Closed
Severity	MEDIUM
File	app_workflow/config.py
Description	All data/feedback/ path constants (FEEDBACK_PATH, USER_THUMBDOWN_PATH, FAILED_VARIANTS_PATH) were originally built as CWD-relative strings rather than resolved against an absolute project root, making them fragile to the directory the process happens to be launched from — FAILED_VARIANTS_PATH had drifted to use one more ../ than the sibling constants.
Root Cause	CWD-relative path construction instead of file_-anchored resolution; services/services.py already used the safer project_root-derived pattern for VECTOR_STORE_PATH and FEEDBACK_PATH, but config.py's own constants had not been brought in line with it.
Solution	Rewrote all data/feedback/ path constants in config.py against a file_-derived PROJECT_ROOT, matching the resolution pattern already used in main.py and services.py, instead of CWD-relative strings.
Date Resolved	2026-06-19

BUG-048 · app/llm_caller.py Missing httpx Exception Handlers and Abort Guard — Out of Parity with app_workflow/services/llm_caller.py

Field	Detail
Issue	app/llm_caller.py's <code>invoke_once()</code> did not handle <code>httpx.HTTPStatusError</code> or <code>httpx.ConnectError</code> , and <code>llm_invoke()</code> 's abort guard was never reachable despite the <code>LLMRateLimitAbortError</code> class and <code>rate_limit_max_delay_seconds</code> parameter already existing in the file
Found Date	2026-06-20
Status	Closed
Severity	HIGH
File	app/llm_caller.py
Description	The June 18 BUG-042 fix added <code>httpx.HTTPStatusError</code> and <code>httpx.ConnectError</code> handlers to <code>app_workflow/services/llm_caller.py</code>, but the equivalent <code>app/llm_caller.py</code> was not updated at the same time. A comparison revealed three gaps: (1) <code>httpx</code> was never imported; (2) <code>_invoke_once()</code> fell through to bare <code>except Exception</code> for <code>httpx.HTTPStatusError/httpx.ConnectError</code>, meaning 429s from the <code>httpx</code> layer were classified as UNKNOWN rather than RATE_LIMIT and never triggered the retry loop correctly; (3) the abort guard in <code>llm_invoke()</code> — <code>if delay >= rate_limit_max_delay_seconds: raise LLMRateLimitAbortError(delay)</code> — was missing despite the class and parameter being present, so <code>app/</code> would continue sleeping through arbitrarily long delays rather than aborting.
Root Cause	BUG-042 was scoped to <code>app_workflow/</code> only; <code>app/llm_caller.py</code> was not updated in the same session.
Solution	Added <code>import httpx as httpx</code> ; added <code>httpx.HTTPStatusError</code> (429 → RATE_LIMIT, 5xx → SERVER_ERROR, other → UNKNOWN) and <code>httpx.ConnectError</code> (CONNECTION) except blocks in <code>_invoke_once()</code> before the bare <code>except Exception</code> ; added the abort guard in <code>llm_invoke()</code> . Both files

	are now fully in parity.
Date Resolved	2026-06-20

BUG-049 · app/api.py Log File Never Created — setup_logging Called After Pipeline Imports

Field	Detail
Issue	Running app/api.py via uvicorn produced no log file under app/run_logs/ , even though setup_logging was called in the file
Found Date	2026-06-20
Status	Closed
Severity	LOW
File	app/api.py
Description	setup_logging(app_name="langchain_api") was invoked after from agent_query import run_agent and other pipeline imports. logger_config.py's setup_logging uses a "configure once" guard — it checks getattr(root_logger, "_CONFIGURED_ATTR", False) and returns immediately if already set. Any earlier import that caused setup_logging to be called first (e.g. agent_query.py or another module that calls it at module load) would win the race, configure the root logger without a file handler aimed at app/run_logs/, and cause api.py's call to be silently skipped. The result was that log output went only to the console with no persistent log file.
Root Cause	Idempotent "configure once" guard in setup_logging combined with api.py calling it after all pipeline imports instead of before.

Solution	Moved <code>setup_logging(log_dir= APP_DIR / "run_logs", app_name="langchain_api")</code> to immediately after <code>logger_config</code> is imported and before any other pipeline module is imported. Log files now appear correctly in <code>app/run_logs/</code> .
Date Resolved	2026-06-20

BUG-050 · app_workflow/nodes/nac.py — caller_tag="MERGE" Ambiguous in DEBUG Logs After Timing Instrumentation

Field	Detail
Issue	<code>merge_similar_chunks()</code> in <code>app_workflow/nodes/nac.py</code> passed <code>caller_tag="MERGE"</code> to <code>llm_invoke()</code> , producing DEBUG log lines tagged <code>[MERGE]</code> — indistinguishable from any other merge-type call if similarly named tags were used in other nodes.
Found Date	2026-06-22
Status	Closed
Severity	LOW
File	<code>app_workflow/nodes/nac.py</code>
Description	After the June 22 addition of per-attempt duration logging to <code>llm_caller.py</code> (logging <code>[{caller_tag}] LLM call attempt N took X.XXXs</code>), the granularity of <code>caller_tag</code> values became directly visible in run logs. The tag <code>"MERGE"</code> used by NAC's <code>merge_similar_chunks()</code> does not identify the calling node, making NAC's LLM calls indistinguishable from any future merge node that might use the same string.
Root Cause	The initial <code>caller_tag</code> value was assigned before DEBUG timing instrumentation existed; tag distinctiveness was not a concern at the time.
Solution	Renamed <code>caller_tag</code> from <code>"MERGE"</code> to <code>"NAC-MERGE"</code> in <code>merge_similar_chunks()</code> so, NAC's LLM calls are unambiguously identified in DEBUG logs.

Date Resolved	2026-06-22
----------------------	------------

BUG-051 · Timeout Constants in Both config.py Files Initially Set Without Empirical Calibration

Field	Detail
Issue	<code>LLM_RESPONSE_TIMEOUT_SECONDS</code> , <code>RETRIEVAL_TIMEOUT_SECONDS</code> , and <code>EMBEDDING_ENCODING_TIMEOUT_SECONDS</code> were initially set based on documentation estimates rather than measured latencies, making the values overly conservative. They were revised after the June 22 benchmark runs revealed the actual p95 figures.
Found Date	2026-06-22
Status	Closed
Severity	LOW
File	<code>app/config.py</code> , <code>app_workflow/config.py</code>
Description	When the three timeout constants were first introduced, no timing baseline existed for this hardware and workload, so documentation figures were used as placeholders. The initial values were larger than warranted — a pathological hang would have been tolerated far longer than necessary before the timeout mechanism triggered.
Root Cause	No empirical timing data was available at the time the constants were introduced; documentation-based estimates were used as a temporary placeholder.
Solution	After running 10 benchmark queries per pipeline with per-module DEBUG timing enabled, empirical p95 latencies were established: LLM calls up to ~75 s; ChromaDB <code>.query()</code> up to ~0.704 s; <code>model.encode()</code> up to ~0.019 s. Constants revised to <code>LLM_RESPONSE_TIMEOUT_SECONDS = 150</code> (2× observed max), <code>RETRIEVAL_TIMEOUT_SECONDS = 10</code> (~14× p95), <code>EMBEDDING_ENCODING_TIMEOUT_SECONDS = 10</code> (~500× p95). Asymmetric margins reflect that LLM latency has higher variance while retrieval and embedding are stable sub-

	second operations.
Date Resolved	2026-06-22

BUG-052 · app_workflow/services/logger_config.py Relative Import Creates Orphaned Timing-Tracker Singleton

Field	Detail
Issue	The JSON timing file produced by each app_workflow/ run remained empty after every query, despite set_json_path() being called at startup and record() being called from every instrumented node.
Found Date	2026-06-23
Status	Closed
Severity	MEDIUM
File	app_workflow/services/logger_config.py
Description	logger_config.py used from .timing_tracker import timing_tracker (a relative import). Python registered this module under the key services.timing_tracker in sys.modules — without the app_workflow. prefix, because the module was loaded as a side-effect of importing logger_config before the app_workflow package namespace was fully established. Node files importing from app_workflow.services.timing_tracker import timing_tracker used the fully-qualified key app_workflow.services.timing_tracker, which is a different cache entry and therefore a different object instance. set_json_path() was called on the orphaned services.timing_tracker instance; all record() calls from nodes went to the uninitialized app_workflow.services.timing_tracker instance, which had no _json_path set and never wrote anything to disk.
Root Cause	Python's import system registers modules under the exact dotted name used in the import statement. A relative import inside a package loaded without its parent fully initialised can produce a sys.modules key that differs from the fully-qualified package.module path used by other importers, yielding two separate objects for the same file.

Solution	Replaced <code>from .timing_tracker import timing_tracker</code> in <code>logger_config.py</code> with <code>import importlib; _tt = importlib.import_module("app_workflow.services.timing_tracker")</code> , then called <code>_tt.set_json_path(json_path)</code> . <code>importlib.import_module</code> always uses the canonical absolute dotted path as the <code>sys.modules</code> key, so it retrieves the same object that node files access via <code>from app_workflow.services.timing_tracker import timing_tracker</code> .
Date Resolved	2026-06-23

BUG-053 · Concurrent POST /query Requests Cause One Thread to Silently Exhaust All Retry Attempts on 429

Field	Detail
Issue	Submitting two simultaneous <code>POST /query</code> requests to <code>app/api.py</code> causes one to silently fail — <code>llm_invoke()</code> returns a rate-limit error after exhausting all retry attempts, every one of which was a 429, because the concurrent thread consumed the available Groq token quota.
Found Date	2026-06-24
Status	Closed
Severity	HIGH
File	<code>app/llm_caller.py</code> , <code>app/api.py</code> , <code>app_workflow/services/llm_caller.py</code> , <code>app_workflow/api.py</code>
Description	When two requests arrive at <code>POST /query</code> simultaneously, <code>asyncio.to_thread()</code> dispatches both <code>run_agent()</code> calls into the thread pool concurrently. Both threads eventually reach <code>llm_invoke()</code> at roughly the same time. The first thread proceeds normally and its LLM calls consume most or all of the Groq token-per-minute quota. Every retry attempt in the second thread hits a 429 rate-limit response. Once <code>LLM_RATE_LIMIT_MAX_ATTEMPTS</code> is exhausted, <code>llm_invoke()</code> returns <code>LLMResult(ok=False, error_kind=RATE_LIMIT)</code> with no successful call. The pipeline produces an erroneous or empty answer — no exception is raised at the HTTP boundary, and no HTTP 429 is returned to the caller. The only evidence is the debug log showing all retry

	attempts hitting 429.
Root Cause	The <code>asyncio.Lock</code> (<code>_query_lock</code>) in <code>api.py</code>'s <code>/query</code> handler appeared to serialize concurrent requests but is not a cross-thread mutex — it operates within a single asyncio event loop thread. <code>asyncio.to_thread()</code> dispatches the blocking work to the thread pool immediately upon entry, releasing the lock before <code>run_agent()</code> begins executing. Both pipeline executions therefore run concurrently inside the thread pool with no mutual exclusion at the <code>llm_invoke()</code> level, where Groq calls actually occur.
Solution	Introduced a FIFO serialization gate inside <code>llm_invoke()</code> in both <code>app/llm_caller.py</code> and <code>app_workflow/services/llm_caller.py</code>: a <code>queue.Queue</code> of per-thread <code>threading.Event</code> objects ensures only one thread calls Groq at a time, in strict arrival order. A thread that receives a 429 holds the gate rather than re-enqueuing, sleeps the full token-reset window from Groq's response headers, then retries directly. The <code>async with _query_lock:</code> block was removed from both <code>api.py</code> files — serialization is delegated to <code>llm_invoke()</code>'s FIFO gate in both pipelines. <code>max_retries=0</code> set on all <code>ChatGroq</code> instances in both packages so LangChain's built-in retry loop is fully disabled. Adaptive cooldown (<code>_cooldown_floor</code>) added to absorb residual token-rate variance between consecutive calls. Branch <code>setup_async_lock</code> merged into <code>master</code> on 2026-06-25 (13 files changed). See ADR-044, ADR-045, and Research topic 35.
Date Resolved	2026-06-25

BUG-054 · IndexKeySpecsConflict on app_workflow/ Startup — sparse=True Mismatch on request_id Index

Field	Detail
Issue	<code>app_workflow/services/db.py</code> raised <code>pymongo.errors.OperationFailure: IndexKeySpecsConflict</code> on startup because it tried to create the <code>request_id_1</code> unique index with <code>sparse=True</code> , while <code>app/db.py</code> had already created that index without <code>sparse=True</code> in the shared <code>rag_db</code> database.
Found Date	2026-06-27
Status	Closed
Severity	HIGH
File	<code>app_workflow/services/db.py</code>
Description	Both <code>app/db.py</code> and <code>app_workflow/services/db.py</code> call <code>ensure_indexes()</code> on the same shared MongoDB database (<code>rag_db</code>). <code>app/db.py</code> creates <code>db["feedback_interactions"].create_index([("request_id", ASCENDING)], unique=True)</code> — no <code>sparse</code> argument, so the index is non-sparse by default. <code>app_workflow/services/db.py</code> initially called the same line with an additional <code>sparse=True</code> . When <code>app/api.py</code> started first, the non-sparse index was registered under the name <code>request_id_1</code> . When <code>app_workflow/api.py</code> started afterward, its <code>ensure_indexes()</code> attempted to create <code>request_id_1</code> with <code>sparse=True</code> — MongoDB rejected this with <code>IndexKeySpecsConflict</code> because <code>sparse</code> is an immutable option that cannot be changed on an existing index without dropping and recreating it. The exception propagated out of the collection accessor, preventing all MongoDB operations and crashing <code>app_workflow/</code> startup.
Root Cause	<code>sparse=True</code> was added to <code>app_workflow/services/db.py</code> to avoid potential unique-index violations from CLI-path interactions that have no <code>request_id</code> . That problem was actually solved in <code>feedback_store.py</code> by generating <code>request_id = request_id</code> or <code>str(uuid.uuid4())</code> , making <code>sparse=True</code> both unnecessary and harmful.
Solution	Removed <code>sparse=True</code> from <code>app_workflow/services/db.py</code> 's <code>request_id</code> index call. Both <code>db.py</code> files now issue identical <code>create_index([("request_id", ASCENDING)], unique=True)</code>

	calls. MongoDB's <code>create_index</code> is idempotent when the spec exactly matches the existing index — the second call is a no-op. See Research topic 36 for background on replica-set index creation.
Date Resolved	2026-06-27

BUG-055 · DuplicateKeyError in feedback_store.log() When LangGraph Retries generate_answer Node

Field	Detail
Issue	After the MongoDB migration, <code>feedback_store.log()</code> raised <code>pymongo.errors.DuplicateKeyError</code> and crashed the <code>generate_answer</code> node when LangGraph's retry machinery re-executed the node within the same graph invocation.
Found Date	2026-06-27
Status	Closed
Severity	HIGH
File	<code>app/feedback_store.py</code> , <code>app_workflow/services/feedback_store.py</code>
Description	LangGraph's <code>run_with_retry</code> can re-execute a node after a transient failure. <code>generate_answer</code> calls <code>feedback_store.log()</code> before returning to the graph. If the node fails after <code>log()</code> but before returning successfully, LangGraph retries it — calling <code>log()</code> again with the same <code>request_id</code> . The <code>feedback_interactions</code> collection has a unique index on <code>request_id</code> . The second <code>insert_one</code> violates this index and raises <code>DuplicateKeyError</code> , which propagates out of the node as an unhandled exception. The flat-file predecessor (<code>interactions.jsonl</code>) was append-only and never raised on a duplicate <code>request_id</code> , so this retry case was never observable before the MongoDB migration.
Root Cause	<code>insert_one</code> had no handler for the scenario where the same <code>request_id</code> was already inserted in the current invocation by an earlier (successful) execution of the same node. The unique index enforces a correctness invariant that is correct at the HTTP-request level but must be treated as an idempotency guard rather than a fatal error at the LangGraph-node level.
Solution	Wrapped <code>interactions_col().insert_one(record)</code> in <code>try/except DuplicateKeyError</code> in both <code>app/feedback_store.py</code> and <code>app_workflow/services/feedback_store.py</code> . On <code>DuplicateKeyError</code> , <code>log()</code> emits <code>logger.warning("Duplicate interaction log skipped (node retry): request_id=...")</code> and returns cleanly — the first insert already persisted the correct record. See Research topic 37 for the general idempotency-via-

	DuplicateKeyError pattern.
Date Resolved	2026-06-27

BUG-056 · fix_llm_output.py Hardcodes model_name="llama-3.1-8b-instant" — Will Break on 2026-08-16 Groq Deprecation

Field	Detail
Issue	<code>_LLM_Json_Repair</code> and <code>Verify_And_Correct</code> in both <code>app/fix_llm_output.py</code> and <code>app_workflow/services/fix_llm_output.py</code> construct fresh <code>ChatGroq</code> instances with <code>model_name="llama-3.1-8b-instant"</code> hardcoded at class instantiation time. Groq has announced <code>llama-3.1-8b-instant</code> will be deprecated on 2026-08-16. After that date, any <code>fix_llm_output.py</code> repair call will fail with a Groq provider error, crashing the LLM output-fix path.
Found Date	2026-06-30
Status	Open
Severity	HIGH
File	<code>app/fix_llm_output.py</code> , <code>app_workflow/services/fix_llm_output.py</code>
Description	Unlike the main pipeline LLM clients (<code>llm</code> , <code>merge_llm</code> , <code>judge_llm</code>) which are instantiated in <code>services/services.py</code> (or <code>agent_query.py</code>) and can be updated at a single config-driven location, the two auto-repair classes in <code>fix_llm_output.py</code> each create their own <code>ChatGroq(model_name="llama-3.1-8b-instant", ...)</code> instance internally. They are therefore not affected by any change to <code>LLM_MODEL_NAME</code> in <code>config.py</code> or by swapping the model reference in <code>services.py</code> . After 2026-08-16, every call to <code>_LLM_Json_Repair.repair()</code> and <code>Verify_And_Correct.verify()</code> will fail at the Groq provider level. The failure surface is the output-fix path — triggered only when the primary LLM returns malformed JSON or a schema violation — so it may go undetected in runs where the primary LLM output is already well-formed, only surfacing under stress or after prompt changes that increase schema violation frequency.

Root Cause	<code>fix_llm_output.py</code> was written before the centralised service-registry pattern (<code>services/services.py</code>) was established. The repair classes take no LLM client as a constructor parameter and instead hard-construct their own. The <code>"llama-3.1-8b-instant"</code> string was never extracted to a config constant.
Solution	Refactor both <code>_LLM_Json_Repair</code> and <code>_Verify_And_Correct</code> in both <code>fix_llm_output.py</code> files to accept an LLM client as a constructor parameter (injected from the service registry) rather than hard-constructing their own. Alternatively, at minimum, extract <code>"llama-3.1-8b-instant"</code> to a <code>FIX_LLM_MODEL_NAME</code> constant in <code>config.py</code> so the model can be changed at config-level before the 2026-08-16 deadline without touching the class logic. The injection-based approach is preferable as it removes the second hidden <code>ChatGroq</code> construction entirely.
Date Resolved	—

BUG-057 · NVIDIA GPU Driver Failure (Code 43) Forces Embedding Fallback to CPU

Field	Detail
Issue	<code>all-MiniLM-L6-v2</code> embedding runs on CPU instead of the RTX 5050 Laptop GPU despite a correctly CUDA-built PyTorch install
Found Date	2026-07-01
Status	Open
Severity	MEDIUM
File	N/A — Windows NVIDIA driver / Device Manager, not project code
Description	<code>embedding_manager.py</code> logged Model 'all-MiniLM-L6-v2' loaded on cpu and CUDA not available; running on CPU. <code>python -c "import torch; print(torch.cuda.is_available())"</code> confirmed <code>torch==2.11.0+cu128</code> (CUDA 12.8 build) but <code>torch.cuda.is_available()</code> returned <code>False</code> . <code>nvidia-smi</code> failed with a permissions error (exit code 4). <code>Get-CimInstance Win32_VideoController</code> showed the RTX 5050 Laptop GPU with <code>Status: Error</code> and <code>ConfigManagerErrorCode = 43</code>

	("Windows has stopped this device because it reported problems"), while the integrated Intel UHD Graphics reported Status: OK . Installed driver version 32.0.15.9595 ($\approx$ NVIDIA 559.95) is suspected too old for proper Blackwell-generation (RTX 50-series) laptop GPU support.
Root Cause	Windows NVIDIA kernel driver in a crashed/error state (Code 43), most likely a driver-version mismatch for the Blackwell-architecture RTX 5050 Laptop GPU. Not a PyTorch or project-code issue — the CUDA-enabled wheel is correctly installed.
Solution	Clean driver reinstall recommended: boot into Safe Mode, run Display Driver Uninstaller (DDU) to fully remove the existing driver, then install the latest Game Ready driver (576.xx+) from nvidia.com for the RTX 5050 Laptop GPU, choosing a clean installation. Re-verify with <code>nvidia-smi</code> and <code>torch.cuda.is_available()</code> after reboot. If Error 43 persists after a clean driver install, check for a BIOS-level GPU MUX/Optimus switch. Not yet performed as of the found date — embeddings currently run on CPU (functionally correct, slower).
Date Resolved	—

BUG-058 · `_LLM_Json_Repair` Cannot Recover Python Class/Attribute-Assignment Syntax — Returns `{}` (Data Loss)

Field	Detail
Issue	When an LLM emits a Python class definition with attribute assignments instead of a dict/JSON literal, <code>fix_llm_output()</code> returns an empty object across every affected schema — in three of four observed cases the LLM repair tier never even fires
Found Date	2026-07-01 (via <code>app_workflow/test_output_fixes.py</code> run, log <code>test_output_fixes_20260701_184529.log</code>)
Status	Open
Severity	MEDIUM
File	<code>app_workflow/services/fix_llm_output.py</code>
Description	Test cases <code>PC04-mj</code> , <code>PC04-rj</code> , <code>PC04-lj</code> (<code>merge_judge</code> , <code>retrieval_judge</code> , <code>lbc_judge</code> schemas) return <code>{}</code> in ~ 0.1 ms —

	meaning the escalation heuristic never recognizes the input as needing the LLM repair tier at all, despite the deterministic tiers (preprocessing, balanced extraction, <code>json_repair</code>) having no ability to parse Python class syntax. <code>PC04-1c</code> (<code>lbc_compress</code> schema) does escalate (671ms, consistent with an LLM repair call being made) but still returns <code>{}</code> — so the LLM repair tier itself also fails to reconstruct the data from class-attribute syntax when it is actually invoked. Test notes for all four cases state the expectation "LLM reads class attribute assignments and emits the JSON object," but none do. This is effectively two separate defects: a missing escalation trigger for this failure shape, and a genuine LLM-repair capability gap once escalation does occur.
Root Cause	None of the deterministic tiers (string preprocessing, balanced-bracket extraction, <code>json_repair</code>) recognize <code>ClassName(): self.field = value</code> syntax as JSON-adjacent, so most cases fall straight through to the empty-result path without ever reaching <code>_LLM_Json_Repair()</code>. Where escalation does happen, the <code>_JSON_REPAIR_PROMPT</code> (schema + raw text) is apparently insufficient to reliably guide reconstruction from this syntax shape, unlike the XML/YAML/dataclass/PascalCase cases it demonstrably does handle (per BUG-018's closure notes).
Solution	Add a preprocessing-tier detection heuristic for <code>class \w+.*:\s*\n\s+self.\w+\s*=</code> patterns (or similar) so these inputs are routed to <code>_LLM_Json_Repair()</code> rather than falling through to the empty result immediately. Separately, strengthen or add a worked example to <code>_JSON_REPAIR_PROMPT</code> specifically demonstrating class-attribute-to-JSON reconstruction, since the existing prompt structure (Research topic 19) evidently does not generalize to this input shape from its current examples.
Date Resolved	—

BUG-059 · merged_from Coercer Only Handles None → 0, Contradicting Its Own Documented Fallback Behavior

Field	Detail
Issue	Pydantic <code>mode="before"</code> coercion for the <code>merged_from</code> field (and likely similarly-documented numeric coercers) is documented/annotated as falling back to <code>0</code> for any bad value, but only actually does so for <code>None</code> — a truncated value becomes <code>""</code> and a non-numeric string passes through unchanged
Found Date	2026-07-01 (via <code>app_workflow/test_output_fixes.py</code> run, log <code>test_output_fixes_20260701_184529.log</code>)
Status	Open
Severity	LOW
File	<code>app_workflow/services/fix_llm_output.py</code>
Description	Test cases <code>C07-merge</code> and <code>C11-merge</code> carry notes claiming the <code>merged_from</code> coercer "falls back to 0" for bad values. Empirically: a truncated numeric value coerces to <code>""</code> (empty string, not <code>0</code>), and the string <code>"two"</code> passes through unmodified (not coerced to <code>0</code>). A separate passing case, <code>C13-merge</code> (a <code>None</code> input), does correctly coerce to <code>0</code> , confirming the <code>None</code> path works as documented — only the empty-string and non-numeric-string paths are missing.
Root Cause	The <code>mode="before"</code> validator for this field (see Research topic 18 for the general coercion pattern) implements only the <code>None → 0</code> branch; it was never extended to cover empty-string or non-numeric-string inputs despite inline documentation/comments implying broader coverage.
Solution	Extend the <code>merged_from</code> (and any sibling numeric-field) coercer's <code>mode="before"</code> validator to also catch empty string and non-numeric string inputs, coercing them to <code>0</code> alongside the existing <code>None</code> case — e.g. <code>if v is None or (isinstance(v, str) and not v.strip().lstrip("-").isdigit()): return 0</code> . Update or remove the inline documentation if the intended behavior is actually narrower than what it currently states.
Date Resolved	—

BUG-060 · LLM Repair Tier Fabricates Plausible JSON for Inputs With No Real Answer Data

Field	Detail
Issue	When the raw LLM response contains no actual answer data at all (bare function definitions, generator/comprehension syntax, f-string templates, lambda-valued dicts, pure prose, or an explicit refusal message), <code>LLM_Json_Repair()</code> still returns a fabricated JSON object instead of signaling extraction failure
Found Date	2026-07-01 (via <code>app_workflow/test_output_fixes.py</code> run, log <code>test_output_fixes_20260701_184529.log</code>)
Status	Open
Severity	MEDIUM
File	<code>app_workflow/services/fix_llm_output.py</code>
Description	Observed across multiple schemas and input shapes: <code>PC01-dc</code> (pure Python function, no dict), <code>PC06-dc/PC09-dc</code> (generator/list-comprehension syntax), <code>PC11-rj</code> (comprehension syntax), <code>PC20-merge</code> (an f-string template with no literal values), <code>PC10-rj</code> (a lambda-valued dict), <code>C01-merge</code> (pure prose, no JSON anywhere), and <code>C18-dc</code> (an outright LLM refusal message). In every case, <code>LLM_Json_Repair()</code> returns a plausible-looking populated object rather than an empty/failure result — meaning downstream consumers receive fabricated values with no signal that the source data never existed. This is functionally identical to the risk already documented for <code>Verify_And_Correct()</code> in BUG-019 (hallucinated <code>sources</code> field), but occurs earlier in the pipeline and across a much broader set of field types, not just <code>sources</code> .
Root Cause	<code>JSON_REPAIR_PROMPT</code> instructs the model to reconstruct JSON matching the target schema from the raw text, but does not give the model an explicit "no data present — return null/empty" escape hatch, nor examples demonstrating when not to produce output. A schema-aware LLM under those instructions defaults to satisfying the schema by inventing plausible values rather than reporting absence.
Solution	Add an explicit negative instruction and worked example to <code>JSON_REPAIR_PROMPT</code> : "If the RAW RESPONSE contains no data corresponding to a required field — no dict/object literal,

	no key-value pairs resembling the schema, only code, prose, or a refusal — return <code>null</code> (or the schema's designated empty/failure marker) rather than inventing values." Cross-reference with the <code>VALUE_VERIFY_PROMPT</code> fix already proposed for BUG-019, since both stem from the same underlying "invent rather than admit absence" failure mode.
Date Resolved	—

BUG-061 · Balanced-JSON Extraction Selects the Wrong Candidate When Multiple JSON-Like Blobs Are Present

Field	Detail
Issue	When a raw LLM response contains both an explanatory/example JSON object and the actual answer JSON object, the extraction tier sometimes grabs the wrong one
Found Date	2026-07-01 (via <code>app_workflow/test_output_fixes.py</code> run, log <code>test_output_fixes_20260701_184529.log</code>)
Status	Open
Severity	LOW
File	<code>app_workflow/services/fix_llm_output.py</code>
Description	Test cases <code>PC14-merge</code> and <code>PC11-lc</code> both include an example/illustrative JSON blob alongside the real answer in the raw response. Extracted output picked up the example values instead of the real ones (e.g. <code>content: 'wrong'</code> instead of the intended <code>'x'</code> ; <code>compressed: 'wrong'</code> instead of the intended <code>'correct'</code>). The balanced-bracket extraction tier (Tier 2, per the <code>fix_llm_output.py</code> module description in <code>Architecture.md</code>) takes the first top-level <code>{...}/[...]</code> it finds, which is correct when there is exactly one candidate object but ambiguous when there are two.
Root Cause	The extraction heuristic has no notion of "which candidate object is the actual answer" — it is positional (first-found) rather than content-aware, so a well-formed example placed before the real answer in the response wins.
Solution	When balanced extraction finds more than one top-level candidate object, prefer the one that best matches the target

	schema's required-field set (e.g. score each candidate by count of expected keys present) rather than defaulting to the first one found. Alternatively, bias toward the last candidate object on the theory that LLMs typically place worked examples before their final answer — would need empirical validation against a broader sample before committing to either heuristic.
Date Resolved	—

BUG-062 · combine_tracks Fires Once Per Predecessor Instead of Waiting for Both Tracks (Non-Barrier Fan-In)

Field	Detail
Issue	combine_tracks executes once for each completing predecessor track rather than once as a true fan-in barrier, because the document and learned-QA tracks have unequal depth
Found Date	2026-07-02
Status	Closed
Severity	HIGH
File	app_workflow/graph.py, app_workflow/nodes/combine_tracks.py
Description	combine_tracks has two incoming edges (validate_LBC_documents, validate_LBC_learned_qa). The document track is NAC → DC → LBC (3 compression stages); the learned-QA track is DC → LBC (2 stages). Because of this depth mismatch the two tracks land in different LangGraph supersteps, and LangGraph's default fan-in only merges predecessors completing in the same superstep — so combine_tracks fired once when the faster learned-QA track finished and again once the slower document track finished. Confirmed in langgraph_api_20260702_140345.debug.log: [COMBINE_TRACKS] learned_qa=4 documents=0 combined=4 immediately followed later by [COMBINE_TRACKS] learned_qa=4 documents=2 combined=6 for the same request. Each premature firing routed straight into generate_draft (since compressed_docs was already non-empty), producing an initial draft built from partial context before the document track had finished compressing — the direct

	cause of the "many drafts/answers per request" and "draft generated before retrieval/compression completed" symptoms observed across that log.
Root Cause	The node was wired as a fan-in point but not registered with the barrier semantics LangGraph requires for predecessors of unequal path length.
Solution	Registered the node with <code>defer=True</code> in <code>graph.py</code> 's <code>add_node("combine_tracks", combine_tracks, defer=True)</code> call — LangGraph now holds execution until every other pending task in the run has completed, so the node fires exactly once with both tracks fully populated.
Date Resolved	2026-07-02

BUG-063 · `generate_answer` Returns draft Verbatim Instead of Using It as Synthesis Input

Field	Detail
Issue	Whenever a draft existed in state, <code>generate_answer</code> returned it unmodified as the final answer instead of using it as working material for one more LLM synthesis call
Found Date	2026-07-02
Status	Closed
Severity	HIGH
File	<code>app_workflow/nodes/generate_answer.py</code>
Description	The intended design (already implemented correctly in <code>app/agent_query.py</code> 's <code>generate_final_answer</code>) is: when a draft exists, pass it as working material into one more LLM call that reconciles it against the full compressed context and produces the polished final answer, falling back to the literal draft <u>only</u> if that synthesis call itself fails. The actual code instead did <code>if draft: answer = draft</code> — a hard short-circuit skipping the synthesis call entirely. Confirmed byte-for-byte in <code>langgraph_api_20260702_140345.debug.log</code> : the <code>[GENERATE_DRAFT] draft: text at 14:11:10</code> and the <code>[GENERATE_ANSWER] RESPONSE: text at 14:11:12</code> are

	identical. This also meant <code>check_answer_quality</code> 's verdict was computed over text that then shipped completely unmodified — judge and answer were the same LLM output with no refinement step between them.
Root Cause	Draft short-circuit was written as the terminal path instead of the fallback path; no synthesis prompt existed for the draft-present case at all.
Solution	Added <code>_GENERATE_ANSWER_FROM_DRAFT_PROMPT</code> (<code>prompts.py</code>) and rewired <code>generate_answer.py</code> : if a draft exists, call the LLM with the draft as working material plus full context to produce the actual answer; fall back to the literal draft only if that call fails (<code>result.ok</code> is <code>False</code>). No-draft path (<code>_GENERATE_ANSWER_PROMPT</code> from flattened context) is unchanged.
Date Resolved	2026-07-02

BUG-064 · GEN_MODEL_NAME Set to a Hugging-Face-Style Model Path While llm Is a ChatGroq Client → HTTP 404 model_not_found

Field	Detail
Issue	<code>.env</code> 's <code>GEN_MODEL_NAME</code> was set to an HF Hub-style model path consumed by the Groq-backed <code>llm</code> client, which has no model by that name
Found Date	2026-07-02
Status	Open
Severity	MEDIUM
File	<code>.env</code> , <code>app_workflow/services/llm_setup.py</code>
Description	<code>.env</code> had <code>GEN_MODEL_NAME=Qwen/Qwen2.5-7B-Instruct</code> , but <code>llm</code> in <code>llm_setup.py</code> is constructed as <code>ChatGroq(model=GEN_MODEL_NAME, ...)</code> — Groq's catalog has no model under that name. Confirmed in log: <code>[NAC-MERGE] NotFoundError — HTTP 404: ... 'The model qwen/qwen2.5-7b-instruct does not exist or you do not have access to it.'</code> Unrelated to the HF-router models added for <code>judge_llm/json_fix_llm</code> on 2026-07-01 (ADR-054/055) —

	<code>GEN_MODEL_NAME</code> feeds the separate, Groq-only main <code>llm</code> instance.
Root Cause	Env value appears to have been copied from the HF-router naming convention (used for <code>judge_llm/json_fix_llm</code>) into a variable consumed by the Groq client, which expects Groq's own model catalog naming.
Solution	Set <code>GEN_MODEL_NAME</code> to a valid Groq model ID. <code>.env</code> now sets <code>GEN_MODEL_NAME=llama-3.1-8b-instant</code> (the invalid <code>Qwen/Qwen2.5-7B-Instruct</code> value is kept as a commented-out line rather than removed).
Date Resolved	2026-07-06 (confirmed fixed in <code>.env</code> ; exact fix date uncertain — <code>.env</code> is gitignored and untracked)

BUG-065 · HF Inference Providers Router Returns HTTP 402 Under Sustained Benchmark Load, Inflating Latency

Field	Detail
Issue	Every benchmark run that routed validators/JSON-fix/CAQ through the HF Inference Providers router recorded at least one HTTP 402 response, and the two setups relying on it were markedly slower than the HF-free setup
Found Date	2026-07-06 (<code>Execution Time Comparison.md</code> benchmark — Setups 1 and 3)
Status	Open
Severity	MEDIUM
File	<code>app_workflow/services/llm_caller.py</code> , <code>app_workflow/services/llm_setup.py</code> (<code>judge_llm / json_fix_llm</code> on the HF router)
Description	Across the 3-setup × 3-query × 2-run benchmark, every run that sent validator/JSON-fix/CAQ traffic to the HF Inference Providers router (Setup 1: those roles on HF; Setup 3: those roles on HF, everything else on Groq) logged at least one HTTP 402 in its <code>Token Limit</code> column in <code>Execution Time Comparison.md</code> . Setup 1's two complex-query averages (29:38, 31:22) were the slowest of all nine runs across all three setups; Setup 2, which

	made zero HF-router calls, logged no 402s and was the fastest setup on every query (2:22 avg best-case). See Research topic 46 for the full comparison.
Root Cause	The HF Inference Providers router's free/shared-tier serving returns <code>402 Payment Required</code> once request volume or context size for the selected model exceeds its free quota within a rolling window — a distinct failure mode from the <code>429</code> rate-limit path already handled by the <code>_groq.*/_openai.*</code> exception blocks in <code>llm_caller.py</code>. It is not yet confirmed whether <code>invoke_once()</code> currently classifies this as a generic <code>openai.APIStatusError/UNKNOWN</code> fallback, nor whether the FIFO-gate retry loop applies a sensible backoff before retrying — a <code>402</code> reflects quota exhaustion, not transient capacity, so retrying without a quota-aware wait (or retrying at all within the same window) can burn the retry budget without ever succeeding.
Solution	Add explicit handling for <code>openai.APIStatusError</code> with <code>status_code == 402</code> in <code>invoke_once()</code>, classified as a new <code>LLMErrorKind</code> (e.g. <code>QUOTA_EXCEEDED</code>) rather than folded into <code>RATE_LIMIT</code>, since exponential backoff is the wrong remedy for quota exhaustion. Consider a longer fixed cooldown or a circuit-breaker (fail fast to <code>UNKNOWN</code> after N consecutive 402s within a run) instead of retrying repeatedly against a router that will not succeed until its quota window resets.
Date Resolved	—

BUG-066 · LBC Fabricates Full Paragraphs From Citation-Only Source Fragments — Caught Only By a Length-Ratio Guard, Not Semantic Validation

Field	Detail
Issue	Given a short, citation-only source chunk (as little as 77 characters), LBC's compression call repeatedly invents a full, plausible-sounding paragraph of unrelated factual content rather than compressing what's actually present
Found Date	2026-07-08 (debug-log audit of <code>langgraph_api_20260708_124129.debug.log</code> , pass 2)
Status	Open
Severity	HIGH
File	<code>app_workflow/nodes/lbc.py</code> , <code>app_workflow/services/prompts.py</code> (<code>_LBC_COMPRESS_PROMPT</code>)
Description	Observed at least 6 times in a single pass: a 77-character source fragment reading only "...parameter for the assessment and treatment of children and adolescents with autism spectrum disorder [Source:...]" (a bare citation, no substantive content) produced a fabricated 1,366-character <code>compressed</code> output inventing a full ASD definition, a "1 in 54 children" prevalence statistic, causes, diagnosis process, treatment options, and disparities — none of which existed in the source. Directly violates the LBC prompt's rule: "Do NOT invent new sentences. Every sentence in your output must come from the original chunk." The same expansion pattern (77→1366, 319→1403, 242→710, 209→935, 152→1466, 285→1196 chars) repeated across the pass. Every instance was neutralized only by the existing over-expansion guard (<code>context_compression.py</code>, ADR-017/BUG-004: reject if <code>len(compressed) > len(original)</code>), which discards the fabricated output and keeps the tiny original fragment — a correct outcome, but a blunt length heuristic rather than a semantic check, so a same-length-but-still-fabricated compression would pass undetected. <code>validate_lbc()</code>, the stage's actual semantic safety judge, never caught any of these — but only because it was itself running as an HTTP-402 fallback (<code>verdict=UNKNOWN</code>) for 100% of calls in this pass; see BUG-065.

Root Cause	<code>LBC_COMPRESS_PROMPT</code> gives the model a citation-fragment chunk with essentially no extractable content and asks it to produce query-relevant compressed text; a task-completion bias appears to make the model synthesize plausible domain content from its own parametric knowledge rather than emit <code>IRRELEVANT</code> or a near-empty result for a chunk with nothing to extract.
Solution	Add an explicit low-content escape hatch to <code>LBC_COMPRESS_PROMPT</code> : "If the chunk contains no substantive sentences to extract (e.g. only a citation, heading, or fragment), output <code>IRRELEVANT</code> rather than generating new content." Since the length guard only catches expansion, add a lightweight semantic backstop independent of <code>validate_lbc()</code> 's availability — e.g. a cheap keyword/entity-overlap check between compressed output and source text — so fabrication is caught even when compressed length stays under the original.
Date Resolved	—

BUG-067 · DC Deletes Sentences Before `validate_redundancy` Can Reject Them — Destructive, Structurally Unrecoverable Redundancy Removal

Field	Detail
Issue	DC applies its proposed sentence deletions to chunks immediately, before <code>validate_redundancy()</code> 's confirm/reject verdict is available; a rejected group's deletion is never rolled back, so a false-positive redundancy match permanently destroys content
Found Date	2026-07-08 (debug-log audit of <code>langgraph_api_20260707_153607.debug.log</code> , an all-local run with zero HF-402 interference, isolating this as a genuine model/architecture defect rather than infra noise)
Status	Open
Severity	HIGH
File	<code>app_workflow/nodes/dc.py</code> , <code>app_workflow/nodes/validate_redundancy.py</code> ,

	<code>app/validators.pyvalidate_redundancy()</code>
Description	In one DC window, the LLM judged a sentence from the disparities essay — "Recent CDC surveillance data reported that ASD was identified far more often among boys than girls, with prevalence among boys about 3.4 times that among girls" — as redundant with an unrelated, more general sentence in another chunk, and DC deleted it (along with a related underrecognition-of-girls sentence). Confirmed via full-log grep that neither sentence reappears anywhere after the deletion point — permanently lost. <code>validate_redundancy()</code> subsequently ran for real (no 402 in this log) and correctly rejected the group, but by then the deletion had already been applied; no code path restores rejected content. The user's query in this run explicitly asked about "associated disparities," making this a direct, user-visible completeness loss in the final answer. A second, independent DC window in the same log proposed 17 similarly nonsensical redundancy pairings (e.g. flagging a definition sentence as a duplicate of a prevalence statistic) — harmless only by coincidence, because the string-replace step found no matching text to remove. Separately, <code>validate_redundancy</code> was itself observed to CONFIRM a group that was arguably not a true duplicate, suggesting the judge's own reliability is not a complete backstop even when it does run.
Root Cause	<code>validate_redundancy()</code> is wired as a logging-only, after-the-fact check with no rollback mechanism. DC's own redundancy-matching prompt uses topic/subject similarity rather than requiring bidirectional semantic entailment — the same underlying weakness already tracked as BUG-003, but this session's evidence shows it causing confirmed, irreversible content loss rather than a theoretical risk.
Solution	Either (a) make DC's deletion conditional on <code>validate_redundancy</code>'s verdict — hold the candidate groups, call the validator first, and only delete groups it confirms (inverting the current fire-then-check order), or (b) if pre-validation ordering is architecturally impractical, snapshot the pre-deletion chunk state per group and restore any chunk whose group is later rejected. Raising DC's own confirmation threshold (BUG-003's original recommendation) reduces the frequency of bad candidates but does not close the structural gap on its own.
Date Resolved	—

BUG-068 · No Quality Gate After generate-answer-from-draft — check_answer_quality Validates the Draft, Not the Actual Final Output

Field	Detail
Issue	<code>check_answer_quality</code> (ANSWER-QUALITY) runs on the pre-synthesis draft; the subsequent <code>generate-answer-from-draft</code> synthesis call (BUG-063's fix) can silently drop content relative to the draft with no re-validation of the text actually returned to the user
Found Date	2026-07-08 (debug-log audit of <code>langgraph_api_20260708_124129.debug.log</code> , pass 2)
Status	Open
Severity	MEDIUM
File	<code>app_workflow/graph.py</code> , <code>app_workflow/nodes/generate_answer.py</code> , <code>app_workflow/nodes/check_answer_quality.py</code>
Description	In the analyzed run, <code>check_answer_quality</code> passed the draft with verdict <code>GROUNDED</code> , <code>unsupported_claims=0</code> . The draft addressed 4 of the user's 6 requested subtopics (definition, diagnosis, treatment, disparities) and included a weak placeholder sentence acknowledging the prevalence gap. The subsequent <code>generate-answer-from-draft</code> synthesis call (BUG-063 — draft is correctly used as synthesis input rather than returned verbatim) produced a final answer that dropped both "causes" and "prevalence" entirely, including the draft's prevalence-gap disclosure, with no substitute disclosure — despite the generation prompt's own instruction to "state coverage honestly when it is partial or insufficient." Because no validator runs after this synthesis step, the drop was never caught anywhere in the pipeline; the user-visible answer is the least-validated text in the entire run.
Root Cause	The RETRIEVE→...→DRAFT→JUDGE→SYNTHESIZE node ordering places the only grounding/completeness judge before the last content-altering LLM call — an ordering gap introduced when <code>generate_answer</code> was changed (BUG-063, 2026-07-02) from "return draft verbatim" to "synthesize a new answer from the draft," without revisiting the quality-gate's placement.
Solution	Either move <code>check_answer_quality</code> to run after <code>generate-answer-</code>

	from-draft instead of before it, or add a lightweight second pass (e.g. a coverage-diff check comparing topics/claims present in the draft vs. the final synthesized answer) so any subtopic or disclosure dropped during synthesis is caught before the answer ships.
Date Resolved	—

BUG-069 · main.py CLI Entry Point Never Initialized Phoenix Tracing

Field	Detail
Issue	The standalone CLI entry point never called <code>setup_phoenix_tracing()</code> , so Phoenix recorded nothing for non-API runs
Found Date	2026-07-09
Status	Closed
Severity	MEDIUM
File	<code>app_workflow/main.py</code>
Description	Only <code>api.py</code> called <code>setup_phoenix_tracing()</code> at module load. Running the CLI (<code>python app_workflow/main.py</code>) never registered an OpenTelemetry tracer provider, so <code>opentelemetry.trace.get_current_span().is_recording()</code> was always <code>False</code> for CLI runs and Phoenix received no spans at all, regardless of any other tracing configuration.
Root Cause	<code>setup_phoenix_tracing()</code> was added only to <code>api.py</code> when Phoenix instrumentation was first introduced; the CLI entry point was not updated in the same change.
Solution	Added a <code>setup_phoenix_tracing()</code> call to <code>app_workflow/main.py</code> , mirroring <code>api.py</code> , placed before <code>setup_logging()</code> .
Date Resolved	2026-07-09

BUG-070 · `_TracingHandler` Filtered Out `DEBUG`-Level Logs Before They Could Reach `LangSmith/Phoenix`

Field	Detail
Issue	The new log-to-trace mirroring handler was set to the console handler's log level (<code>INFO</code>), so the majority of node-level <code>logger.debug(...)</code> calls never reached either trace backend
Found Date	2026-07-09
Status	Closed
Severity	MEDIUM
File	<code>app_workflow/services/logger_config.py</code>
Description	<code>_TracingHandler</code> (added to mirror <code>logging</code> records onto the active <code>LangSmith</code> run / <code>Phoenix</code> span) was set via <code>tracing_handler.setLevel(console_level)</code> , and <code>console_level</code> defaults to <code>INFO</code> . A grep across <code>app_workflow/nodes/*.py</code> found 67 of 127 <code>logger.*</code> calls are <code>logger.debug(...)</code> — the majority of per-node pipeline detail was silently dropped before ever reaching the mirroring handler.
Root Cause	The tracing handler's level was tied to the console handler's level instead of being independent. <code>DEBUG</code> -level suppression is appropriate for the terminal (to avoid third-party library noise — see <code>logger_config.py</code> 's existing <code>INFO</code> console level), but not appropriate for the trace-mirroring path, which only mirrors this project's own loggers.
Solution	<code>_TracingHandler</code> now runs at <code>logging.DEBUG</code> unconditionally, independent of <code>console_level</code> . Console and file handler behavior is unchanged.
Date Resolved	2026-07-09

BUG-071 · Phoenix Span Lookup via `opentelemetry.trace.get_current_span()` Always Returns a Non-Recording Span

Field	Detail
Issue	The standard OTel "current span" lookup always returned a non-recording no-op span under Phoenix's LangChain instrumentation, so no log event was ever attached to a Phoenix span
Found Date	2026-07-09
Status	Closed
Severity	HIGH
File	<code>app_workflow/services/logger_config.py</code> , <code>app_workflow/services/phoenix_tracing.py</code>
Description	<code>TracingHandler</code> 's first implementation looked up the "current" Phoenix span via <code>opentelemetry.trace.get_current_span()</code> , following standard OTel usage. In every real run (confirmed via temporary <code>[trace-debug]</code> print statements added to the handler), this returned <code>NonRecordingSpan(...)</code> (<code>is_recording() == False</code>) for the entire pipeline, while the equivalent LangSmith lookup (<code>get_current_run_tree()</code>) worked correctly for the same call sites in the same run.
Root Cause	Source inspection of <code>openinference-instrumentation-langchain</code> 's <code>OpenInferenceTracer</code> (<code>tracer.py</code>) found it never calls <code>context.attach()</code> or <code>tracer.start_as_current_span()</code> — the calls that would populate OTel's ambient "current span" context var. Instead it stores every span it creates in an internal <code>self._spans_by_run: Dict[UUID, Span]</code> dict keyed by LangChain's own <code>run_id</code> , exposed only via <code>OpenInferenceTracer.get_span(run_id)</code> . This is an undocumented internal implementation detail of the package, confirmed only by reading its source — not a project-introduced defect.
Solution	<code>TracingHandler</code> 's Phoenix branch now reuses the <code>run_id</code> obtained from the (already-working) LangSmith <code>run_tree</code> lookup and calls <code>LangChainInstrumentor().tracer.get_span(run_id)</code> to retrieve the matching Phoenix span directly from its internal registry, then calls <code>.add_event(...)</code> on it. Verified end-to-end: a <code>logger.info(...)</code> call made inside a traced <code>RunnableLambda</code> was

	confirmed to appear in the span's <code>.events</code> list.
Date Resolved	2026-07-09

BUG-072 · LangSmith Web UI Does Not Render Custom Run Events Anywhere (Platform Limitation, Not a Code Defect)

Field	Detail
Issue	Log events mirrored onto LangSmith runs via <code>RunTree.add_event</code> are fully persisted server-side but never rendered anywhere in the LangSmith web UI on this account/plan
Found Date	2026-07-09
Status	Closed (worked around; not fixable in this codebase)
Severity	LOW
File	N/A (LangSmith hosted UI); mitigated in <code>app_workflow/services/trace_events.py</code>
Description	After confirming <code>TracingHandler</code> successfully attaches log records as events on the active LangSmith run, the events were not visible anywhere in the LangSmith web UI — not on the trace's Feedback/Input/Output tabs, not on a node's "Attributes" tab as expandable JSON, and not as markers on the waterfall timeline. Queried the LangSmith API directly (<code>Client().read_run(run_id)</code>) for a real run from a completed trace and confirmed 43 mirrored log events were fully persisted server-side, in order, with full messages — ruling out a code-side bug entirely.
Root Cause	LangSmith's hosted UI (on this account/plan) has no rendering surface for custom <code>events</code> on a run, even though the API/backend accepts and stores them.
Solution	Not fixable from this codebase. Wrote <code>app_workflow/services/trace_events.py</code> as a mitigation — a CLI (<code>python -m app_workflow.services.trace_events <run_id></code>) that resolves a run's <code>trace_id</code> , fetches every run in that trace via the LangSmith API, and prints all mirrored log events in chronological order with their originating node name, bypassing

	the UI gap entirely.
Date Resolved	2026-07-09

BUG-073 · trace_events.py Crashes on Windows Console With UnicodeEncodeError on Non-cp1252 Characters

Field	Detail
Issue	The new <code>trace_events.py</code> CLI crashed partway through printing whenever a mirrored log message contained a character outside Windows' default <code>cp1252</code> console encoding
Found Date	2026-07-09
Status	Closed
Severity	LOW
File	<code>app_workflow/services/trace_events.py</code>
Description	First real run of the CLI crashed with <code>UnicodeEncodeError: 'charmap' codec can't encode character '≈'...</code> — the Windows terminal's default <code>cp1252</code> code page cannot render characters such as <code>≈</code> , em-dashes, and arrows that appear in mirrored log messages (e.g. compression-stage arrows, approx-symbols in stats output).
Root Cause	Python's default stdout encoding on Windows is the console code page (<code>cp1252</code>), not UTF-8; <code>print()</code> raises instead of substituting when a character falls outside that code page.
Solution	Reconfigured the script's stdout stream to UTF-8 with error-tolerant encoding before printing any event line, so out-of-range characters no longer crash the script.
Date Resolved	2026-07-09

BUG-074 · Circular Import in nodes/generate_answer.py — Mixed Absolute/Relative Import Styles Load nodes Package Under Two Identities

Field	Detail
Issue	nodes/generate_answer.py's one intra-nodes cross-import uses the absolute app_workflow.nodes.X form while every other file in the package (and nodes/ __init__.py itself) uses the bare nodes.X form, causing ImportError: cannot import name 'generate_answer' from partially initialized module 'nodes.generate_answer' whenever graph.py/main.py builds the graph
Found Date	2026-07-10
Status	Closed
Severity	HIGH — blocks graph construction via main.py's import path; latent risk regardless of entry point
File	app_workflow/nodes/generate_answer.py, app_workflow/nodes/ __init__.py
Description	Both app_workflow/ and the project root sit on sys.path, so the nodes package can be imported under two distinct module identities: nodes (bare) and app_workflow.nodes (absolute). nodes/ __init__.py and every other node file reference sibling modules as nodes.X, but generate_answer.py:10 used from app_workflow.nodes.check_answer_quality import QUALITY_PASS_VERDICT. Because nodes/ __init__.py imports generate_answer before check_answer_quality in its own import order, the absolute-path import re-enters the still-initializing nodes package under its other identity and fails to find generate_answer defined yet. Reproduced against unmodified master (git stash + rerun) to confirm the defect predated this session's changes — it was latent because main.py's import path triggers it reliably, while api.py (run via uvicorn with app_workflow/ as cwd) happened not to trigger it in this session's testing.
Root Cause	Mixed absolute (app_workflow.nodes.X) vs. bare-package (nodes.X) import styles for the one intra-nodes cross-reference in the codebase, causing the nodes package to load under two distinct module identities depending on import order.

Solution	Changed <code>generate_answer.py:10</code> to <code>from nodes.check_answer_quality import QUALITY_PASS_VERDICT</code> , matching the style used by every other file in the package.
Date Resolved	2026-07-10

BUG-075 · Langfuse CallbackHandler Was a No-Op — config Never Reached Any LLM Call

Field	Detail
Issue	Passing <code>config={"callbacks": [langfuse_handler]}</code> to <code>rag_app.invoke(...)</code> produced zero Langfuse traces — the callback never reached any actual LLM call anywhere in the graph
Found Date	2026-07-10
Status	Closed
Severity	HIGH — Langfuse tracing was silently a complete no-op; the pipeline ran and answered normally with no errors, masking the failure
File	<code>app_workflow/services/llm_caller.py</code> (root fix), plus ~10 files at ~40 call sites: <code>services/fix_llm_output.py</code> , <code>services/validators.py</code> , <code>services/self_learner.py</code> , and node files <code>query_variants.py</code> , <code>check_answer_quality.py</code> , <code>generate_answer.py</code> , <code>generate_draft.py</code> , <code>dc.py</code> , <code>lbc.py</code> , <code>nac.py</code> , <code>validate_retrieval.py</code> , <code>dedup_merge.py</code> , <code>auto_distillation.py</code> , <code>commands.py</code>
Description	Unlike LangSmith (env-var ambient tracing) and Phoenix (OTel auto-instrumentation patched globally at import time), Langfuse's <code>langchain.CallbackHandler</code> requires the LangChain callbacks list to actually reach each <code>.invoke()</code> call. LangGraph only auto-injects <code>config</code> into a node function if that function's signature declares a <code>config</code> parameter; none of the project's ~14 node functions did. Even had that been fixed, <code>services/llm_caller.py</code> 's <code>llm_invoke()/_invoke_once()</code> never accepted or forwarded a <code>config</code> argument to <code>llm.invoke(...)</code> , and <code>_invoke_once()</code> 's <code>ThreadPoolExecutor</code> -wrapped <code>future.result(...)</code> call would have broken ambient contextvar-based propagation regardless of node-level wiring — confirmed by direct inspection

	that a plain <code>BaseCallbackHandler</code> passed the same way also failed to receive events.
Root Cause	The project's node functions and <code>llm_invoke()/invoke_once()</code> were written before any callback-based (as opposed to ambient) tracing backend was in use, so no code path existed to carry an explicit <code>config/callbacks</code> object from the top-level <code>.invoke()</code> call down to the actual <code>llm.invoke(...)</code> calls nested several layers deep across nodes, validators, and repair functions.
Solution	Added an optional <code>config</code> parameter (type <code>RunnableConfig</code>) to every node function's signature and threaded it into the corresponding service-layer calls; updated <code>llm_caller.py</code> 's <code>llm_invoke()/invoke_once()</code> to accept <code>config</code> and forward it explicitly as <code>llm.invoke(messages, config=config, **kwargs)</code> . Verified end-to-end: a live query produced a full Langfuse trace with populated <code>GENERATION</code> observations (model name, input/output, token usage) for every LLM call in the graph, confirmed via Langfuse's REST API (<code>/api/public/observations?type=GENERATION</code>).
Date Resolved	2026-07-10

BUG-076 · Langfuse Silently Evicts Phoenix's Span Exporter From the Shared Global TracerProvider

Field	Detail
Issue	Initializing Langfuse's <code>CallbackHandler</code> while Phoenix's OTel tracer provider is already the process-global default silently removes Phoenix's span exporter from that shared <code>TracerProvider</code> ; once Langfuse is active, Phoenix receives zero further spans regardless of <code>ENABLE_PHOENIX_TRACING</code>
Found Date	2026-07-10
Status	Open — root cause fully diagnosed and a fix verified against isolated throwaway scripts driving the real <code>build_graph()</code> pipeline; not yet applied to <code>phoenix_tracing.py/langfuse_tracing.py</code> , per explicit instruction to research and plan only this round
Severity	HIGH — defeats the "Phoenix primary, run in parallel" intent of ADR-063 the moment Langfuse is also enabled; the two

	backends are not actually independent when both <code>ENABLE_PHOENIX_TRACING</code> and <code>ENABLE_LANGFUSE_TRACING</code> are on
File	<code>app_workflow/services/phoenix_tracing.py</code> , <code>app_workflow/services/langfuse_tracing.py</code>
Description	<code>phoenix.otel.register()</code> installs a <code>SimpleSpanProcessor</code> on the process's global <code>TracerProvider</code> and flags it internally as a replaceable "default" processor (<code>_default_processor = True</code>; Phoenix's own startup log warns "Using a default SpanProcessor. <code>add_span_processor</code> will overwrite this default."). Langfuse's SDK (v3+, installed 4.13.2) is itself OTel-native: <code>get_langfuse_handler()</code> → <code>CallbackHandler()</code> → <code>_client/resource_manager.py</code>'s <code>init_tracer_provider()</code> detects the existing global <code>TracerProvider</code> (Phoenix's) and correctly reuses it rather than creating a separate one, then calls <code>tracer_provider.add_span_processor(langfuse_processor)</code> on it. Because that provider is Phoenix's <code>phoenix.otel.TracerProvider</code> subclass, <code>add_span_processor()</code> is overridden to check <code>self._default_processor</code>; since it is still <code>True</code>, the override calls <code>self._active_span_processor.shutdown()</code>, empties <code>_span_processors</code>, and only then appends the Langfuse processor — deleting Phoenix's exporter as a side effect of Langfuse politely trying to share the existing provider. Confirmed directly by inspecting the live processor list before/after <code>get_langfuse_handler()</code>: (<code><phoenix.otel.SimpleSpanProcessor ...></code>,) → (<code><langfuse._client.span_processor.LangfuseSpanProcessor ...></code>,). No exception is raised anywhere in this chain — /query requests and the graph itself run normally, so the failure is invisible without inspecting OTel internals directly.
Root Cause	Phoenix's <code>TracerProvider.add_span_processor()</code> override treats its own installed processor as disposable ("default", safe to replace) the first time any other library calls <code>add_span_processor()</code> on the shared global provider; Langfuse's SDK calls exactly that method as part of its normal (and otherwise correct) "reuse the existing global provider" behavior.
Solution	Verified but not applied: keep Phoenix off the global provider slot and pass it explicitly to the LangChain instrumentor instead of relying on provider-sharing — <code>register(project_name=..., endpoint=..., set_global_tracer_provider=False)</code> followed by <code>LangChainInstrumentor().instrument(tracer_provider=tp)</code> . With <code>set_global_tracer_provider=False</code> ,

	<code>opentelemetry.trace.get_tracer_provider()</code> returns a <code>ProxyTracerProvider</code> rather than Phoenix's real provider, so when Langfuse's <code>init_tracer_provider()</code> looks for an existing global provider it finds none and creates its own separate one instead of touching Phoenix's, eliminating the eviction path entirely. Verified end-to-end against the real <code>build_graph()</code> pipeline with a live Langfuse callback attached: both Phoenix (confirmed via Phoenix's REST API) and Langfuse received their expected spans from the same run.
--	---

BUG-077 · Marker/pdftext Multi-Worker Extraction Aborts the Whole Conversion on PDFs Over ~40 Pages

Field	Detail
Issue	With Marker's default multi-worker text extraction, a single dead <code>pdftext</code> worker aborts the entire PDF→Markdown conversion on documents over roughly 40 pages, so long PDFs fail to convert at all
Found Date	2026-07-24
Status	Closed (mitigated by configuration)
Severity	MEDIUM — silently fails long-document ingestion; short PDFs are unaffected, so the failure only appears once a large file is ingested
File	<code>marker_service/server.py</code> (<code>_CONVERTER_CONFIG</code>)
Description	<code>pdftext</code> sizes its extraction process pool as <code>min(pdftext_workers, pages // 10)</code> with a provider default of 4 workers. On a document long enough to spin up multiple workers (roughly >40 pages, i.e. $\text{pages} // 10 \geq 4$), if any one worker dies mid-extraction the whole conversion aborts rather than degrading gracefully. Marker's own CLI and server front-ends work around this by forcing a single extraction worker. The isolated Marker microservice reproduces that behaviour so long PDFs (e.g. multi-page clinical/engineering source documents in <code>data_files/</code>) convert reliably. Note: <code>server.py</code> and <code>marker_service/README.md</code> refer to this informally as "BUG-025 in the loader docs" — that is a session-local number predating this catalog and collides with the existing BUG-025 (output-repair flags); the canonical entry is this one, BUG-077.

Root Cause	pdfextract's default multi-worker extraction pool has no per-worker fault tolerance — one worker's death aborts the aggregate conversion instead of retrying or continuing single-threaded — and Marker does not override that default when a PdfConverter is constructed directly (only its CLI/server front-ends do).
Solution	Force pdfextract_workers=1 in marker_service/server.py's _CONVERTER_CONFIG, passed into PdfConverter(config=_CONVERTER_CONFIG), matching Marker's own CLI/server behaviour. Single-worker extraction removes the dead-worker abort path at the cost of slower extraction on very large documents — acceptable because conversions are already serialised behind a threading.Lock (one converter per process) and run on the GPU.
Date Resolved	2026-07-24
Date Resolved	—